Embedded Systems

Repository: Codeberg

Contents

Lesson 1: Introduction **3**
- Embedded system definition 3
- Hardware basics 3
- Embedded system architectures 4
  - `ASIC` - Application-Specific Integrated Circuit 4
  - `ASSP` - Application-Specific Standard Product 4
  - `PLD` - Programmable Logic Device 4
  - `CPLD` - Complex PLD 5
  - `FPGA` - Field Programmable Gate Array 5
  - `MCUs` - MicroController Unit 6
  - `DSP` - Digital Signal Processor 7

Lesson 2: Digital I/O **8**
- Working with ports 8
  - Digital IO port register 8
  - Multiplexed I/O pins 8
  - PinMode write 9
  - Data Latch change (write) 9
  - Switch to PinMode read 9
  - DigitalRead 9

Lesson 3.1: Oscillator **10**
- Oscillator Circuit for dsPIC33 10
  - Clock System Architecture: 10
  - F_{OSC} and F_{CY} 10
  - F_{OSC} and F_{IN} 11

Lesson 3.2: Timers **12**
  - Counter and Gated Mode 12
  - Timer registers 13

Lesson 4.1: Interrupts **15**
- ISR (Interrupt Service Routine) 15
- Interrupts: How They Work 15
  - The Interrupt Vector Table (IVT) and ISR Mapping 16
  - Where to find ISR Names? 16
- Nested Interrupts 17
- Interrupt Flag Status Registers 17
- Interrupt Enable Control Registers 17
- Interrupts and Shared Variables 18
- Critical Regions 18

Lesson 4.2: Peripheral Pin Select (PPS) **19**
- Architectural Overview: Peripheral Pin Select (PPS) 19
- Hardware Implications and Constraints 19
- Implementation Mechanism 19
  - Input mapping example: Assigning External Interrupt 1 to the Left Button 20
  - Output mapping example: 21

Debouncing pattern: Interrupts and PPS **22**

Lesson 5.1: Introduction to Communication Protocols **23**
- The OSI Reference Model 23
- Line drivers 23
- Differential signaling 23
- Simplex - Half-Duplex - Duplex communication 23

Synchronous and Asynchronous communication 24
Serial - Parallel communication 24

Lesson 5.2: dsPIC33 UART Architecture and Configuration 24
UART Transmitter and Receiver Mechanics 24
Control and Status Registers 25
Managing Overrun Errors (OERR) 25
Mode Registers 25
Interrupt Event Selection 26

dsPIC33 UART echo implementation 27

Lesson 6: SPI 32
Overview 32
Applications 32
dsPIC33F SPI Module 33
SPIxBUF Register 33
SPIxSTAT Register 33
SPIxCON1 Register 33
SPI Overflow 34
SPI Example 34
`BOSCH BMX055` SPI interface 34

Lesson 7: ADC 35
Overview 35
dsPIC33 ADC Modules 35
Timing Parameters 35
ADC Configuration Registers 36
ADC Triggers 36
manual sample - manual conversion 36
manual sample - automatic conversion 37
automatic sample - manual conversion 37
automatic sample - automatic conversion 37
ADC Multiple Channels 38
`ADxCON1` Register 38
`ADxCON2` Register 38
`ADxCON3` Register 39
`ADxCHS0` Register 39
`ADxCHS123` Register 39
Other Registers 39
Scan Mode 40
`ANSELx` Register 40
Configuring the ADC 40
DONE bit 40

Appendix 1: Installation of the required Tools 41
Install the Mikrobootloader 41
Install MPLAB in VS Code 41

Appendix 2: Working Pipeline 41
Step 1: Create a Workspace and Project 41
Step 2: Switch from Standard Desktop C environment to Embedded C environment 41
Step 3: Build the Project 42
Step 4: Get product and vendor id 42
Step 5: Flash the Program onto the uC 42

Lesson 1: Introduction

Embedded system definition

> *"An embedded system is a special-purpose computer system designed to perform one or a few dedicated functions, often with real-time computing constraints. It is usually embedded as part of a complete device including hardware and mechanical parts"*
>
> — **Wikipedia**

Embedded Systems, compared with general purpose computers, are *application-oriented cost-sensitive highly-optimized* devices

Positive aspects:	Example of possible requirements
• Lower cost • Higher reliability • Smaller size • Reduced power consumption • Less produced heat	• Small-size devices • Difficult environmental conditions • Portable devices • Reliability devices • On-line failure-detection • Absence human intervention

Hardware basics

PCB: (Printed Circuit Board) Board used for realizing electronic circuits. It is made of dielectric layers laminated with epoxy resins and provides conductive paths and mechanical support to components.

Electronic Components: Basic electronic elements of various nature, constituting electronic circuits, packaged in a discrete form with two or more connecting pads (or pins)

Integrated Circuit (CHIP): Miniaturized electronic circuit contained inside a single component.

General characteristics are:
- Nature: analog, digital, mixed
- Package: SIP (Single-In-line Package), DIP (Dual-In-line Package), QFP (Quad Flat Package), PGA (Pin Grid Array)
- Mounting technique: through holes, sockets, SMD (Sourface Mounting Devices)

Logic Gate: The basic building block of any processing device implementing a logic function

Main features:
- Can be implemented with different transistor technologies (CMOS, TTL, ...).
- Have at least one (usually more) input and one output
- Input and output values are the logical values true `1` and false `0`
- Have no memory (i.e. the output depends only on the current value of the inputs)
- Can be described through a truth table. If n is the number of inputs of a logic gate, its truth table has $2n$ rows and $n + 1$ columns

Basic memory elements
Latch, Flip-flop: circuit elements whose output depends not only on the current inputs, but also on previous inputs and/or outputs. Main features:
- many kinds of latches and flip-flop do exist (SR, D, T, JK, ...)
- they can be synchronous or asynchronous
- they are made of basic gates
- with feedback signals
- can not be described through a truth table

> **[!]** Clock is introduced

Embedded system architectures

`ASIC` - Application-Specific Integrated Circuit

An integrated circuit specifically designed for a special single purpose/application.
- It is not a general-purpose device (application specific)
- ASIC is built for one and only one customer.
- The intellectual property, design database, and deployment of an ASIC are usually controlled by just a single entity (i.e. it is not available to the public.)

`ASSP` - Application-Specific Standard Product

An integrated circuit specifically designed for a special application market.
- It is not a general-purpose device (application specific).
- ASSP is built for more than one customer (standard product)
- The intellectual property, design database, and deployment of an ASIC are available to a certain level

Characteristics of ASICs and ASSPs
- Fixed Hardware: Internal logic is fixed by manufacturers.
- No Programmable Software: The IC may include sequential elements (like microcontrollers) but their instructions are fixed (firmware) and not modifiable by user (apart from updates, eventually available from manufacturers)

Main pros:	Main cons:
Design optimized for the application: • number of gates tailored to the application needs • reduced power consumption • reduced IC size • improved performances Low costs for replica production (they just "contain" what needed)	High design costs (hundreds of k€) for producing IC masks (an IC mask is a glass plate with a pattern of transparent and opaque areas used to photolithographically create patterns on wafers) Low flexibility: no option for circuit modification during fabrication life-cycle of an ASIC/ASSP linked with life-cycle of target products

Final conclusion on ASIC and ASSP
Fixed logic nonprogrammable devices (ASICs and ASSPs) are optimal solutions for high-volume productions. They can be found inside several objects of common use (consumer electronics, house appliances,...)

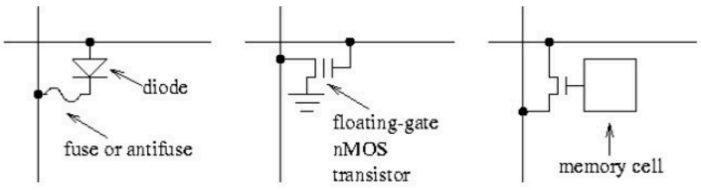
For medium- or small-volume productions a better (i.e. more cost- effective) solution is using standard components, such as small sequential devices (like DSPs or microcontrollers) or hardware- reconfigurable devices (like PLDs or FPGAs).

`PLD` - Programmable Logic Device

An integrated circuit whose internal logic is not rigid but can be configured to some extent according to the user needs.
- PLDs are standard products sold as COTS (commercial-off-the-shelf).
- A PLD has an undefined function at the time of manufacture and before it can be used in a circuit it must be programmed (i.e. internally configured).
- PLDs have been developed since 70s. Originally, they were used for implementing glue logics (i.e. custom electronic circuitries interfacing two or more different ICs).

They are often preferred to sets of gates, since they allow to reduce the number of employed components, save PCB space and are generally more power-efficient

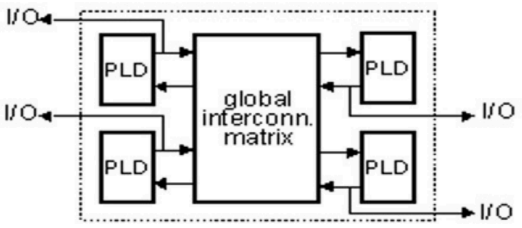
PROM technologies



- OTP: One Time Programmable (fuse or antifuse)
- MTP: Multiple Times Programmables (floating-gate transistor, SRAM,Flash)

`CPLD` - Complex PLD

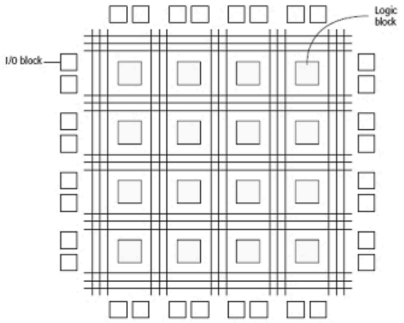
A PLD made up of contains a bunch of PLD blocks whose inputs and outputs are connected by a global programmable interconnection matrix



Two levels of programmability
- each PLD block (logic block)
- the interconnections between the PLDs (Programmable Interconnection Matrix, PIM)

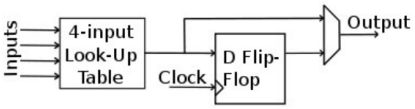
`FPGA` - Field Programmable Gate Array

A PLD made up of a bunch of simple, configurable logic and I/O blocks arranged in an array with interspersed switches that can rearrange the interconnections between the logic and I/O blocks.



FPGA has is a “fine-grain” device
- FPGA has two levels of programmability:
- each logic and I/O block
- the interconnections between blocks

Each logic block has (at least):
- a 4- or 6-input LUT
- 1 flip-flop
- a mux for selecting combinatorial or registered output Logic blocks can be interconnected in different ways



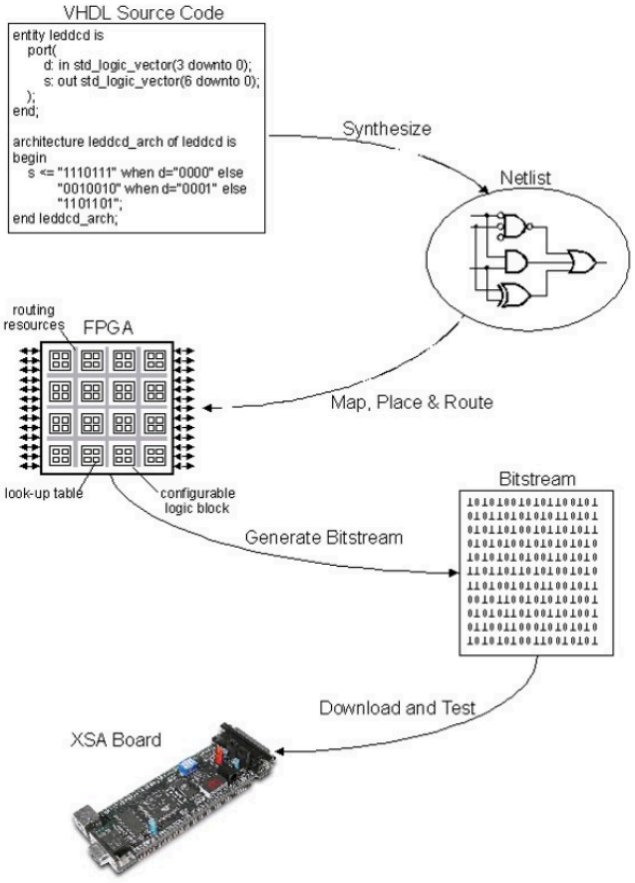


Figure 1: FPGA pipeline

FPGAs PROs and CONs

Pros	Cons
• Lower design costs and lower design risks than with ASICs/ASSPs. • Flexibility. • Highly parallel processing (hundreds/thousands of “operations”). • High level of SoC (system-on-chip) integration (built-in features can reduce the number of components on the board). • High density (top FPGAs have 20k six-input logic functions). • High performance (modern FPGAs operate on frequencies of as high as 550 MHz i.e. only few times less than typical custom ASIC top frequency).	• Relatively power-hungry devices • Higher costs (from around one hundred to thousands dollars) than ASICs/ASSPs for high-volume production • For “simple” applications lower-cost devices do exist • For control-oriented applications MCUs and DSPs may be better solutions

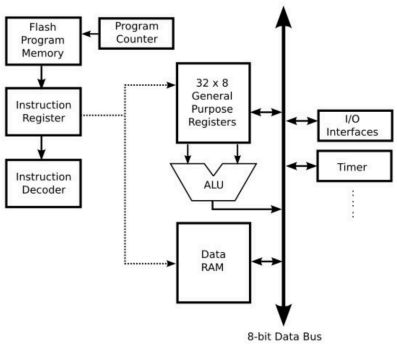
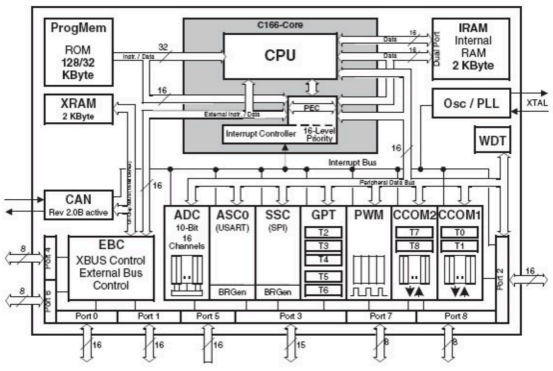
Conclusion on CPLDs and FPGAs

Feature	**FPGA**	**CPLD**
Logic Blocks	Up to 100,000 (tiny blocks)	A few to a few thousand
Architecture	Fine-grain	Coarse-grain
Memory Base	RAM-based	EEPROM-based
Signal Delays	Less predictable	Highly predictable
Cost	Generally more expensive	Much cheaper

`MCUs` - MicroController Unit

A single-chip microcomputer; i.e. a small computer on a single IC consisting of a relatively simple CPU combined with internal memory blocks and I/O control units.

- Unlike PLDs, MCUs have fixed wiring (i.e. the connections between its constituent gates are fixed)
- MCUs are sequentially programmable devices (i.e. they are re-programmed by changing a sequence of fixed operations, instructions, stored in memory)
- In contrast to general-purpose microprocessors, MCUs are simpler, lower-cost, lower-size and less power-hungry.
- Microcontrollers are primarily used in control-oriented applications (often interrupt- driven), for sensing and controlling external events

8-bit MCU	bigger MCU
	

Typical MCUs Resources

• CPU core (ranging from small and simple 4-other serial communications interfaces bit processors to complex 32- or 64-bitlike I²C, SPI and CAN processors) • CPU registers and SFRs (Special Function Registers) for controlling other peripherals • volatile memory (RAM) for data storage • timers	• watchdog • non-volatile memory (ROM, EPROM, EEPROMclock generator circuitry (driven by or Flash memory) for program storageinternal oscillators or external crystals) • programmable GPIO pins and specialADC and DAC • functions pinsin-circuit programming and debugging • serial I/O such as serial ports (UARTs)support • PWM generators

Programming MCUs

- MCUs typically do not have a FPU or a math coprocessor, so both fixed point and floating-point arithmetic are performed by program code.
- Peripherals programming skills are required (i.e. a deeper hardware knowledge than what needed for desktop applications)
- Event-driven programming skills
- MCUs were originally programmed only in assembly language.
- Various high-level programming languages (like C) are now also in common use to target microcontrollers.
- High-level compilers typically have some restrictions as well as enhancements to better support the unique characteristics of microcontrollers.
- An MCU instruction set usually has many instructions intended for bit-wise operations. (For example, a general-purpose processor might require several low-level instructions to test a bit in a register and branch if the bit is set, where a MCU could have a single instruction to provide that commonly-required function.)

`DSP` - Digital Signal Processor

A type of fast and powerful microprocessor whose architecture is optimized for performing intense "number crunching" applications

- Like MCUs, DSPs have fixed wiring (i.e. the connections between its constituent gates are fixed)
- DSPs are sequentially programmable devices (i.e. they are re-programmed by changing a sequence of fixed operations, instructions, stored in memory)
- In contrast to MCUs, DSPs have an architecture optimized for intensive number crunching operations.
- DSPs have typically fewer on-chip peripherals than MCUs.
- DSPs are primarily used in systems that require the precision processing of analog or digital signals.

DSPs Peculiarities

- fast multipliers (dedicated circuitry for MAC (multiply accumulation))
- multiple execution units (such as ALUs working in parallel with MAC units)
- special SIMD (single instruction, multiple data) operations
- circular buffers
- efficient memory access
- Harvard or mod-Harvard architecture
- more data-buses
- dedicated AGU (Address Generation Units)
- few-bit data format (16 to 32 bit architecture with accumulator registers)
- efficient looping
- streamlined I/O (low-latency interrupt, on-chip DMAs)
- deep instruction pipeline
- floating point operations (optionally)
- multicore (optionally)

Conclusion on MCU vs DSP

MCU	DSP
• General-purpose embedded controller • Rich peripherals (ADC, UART, SPI, CAN, computation timers) • Good for control logic + system • Lower cost and power • Suitable for kHz-range control loops	• Designed for filtering, FFT, motor control, power electronics management • Fast MAC units, SIMD, Harvard architecture • Optimized for numerical • Deterministic high-frequency loops • Higher computational throughput

Lesson 2: Digital I/O

In the following schematics active-low logic convention is used: Logic High is 0 and Logic Low is 1

Working with ports

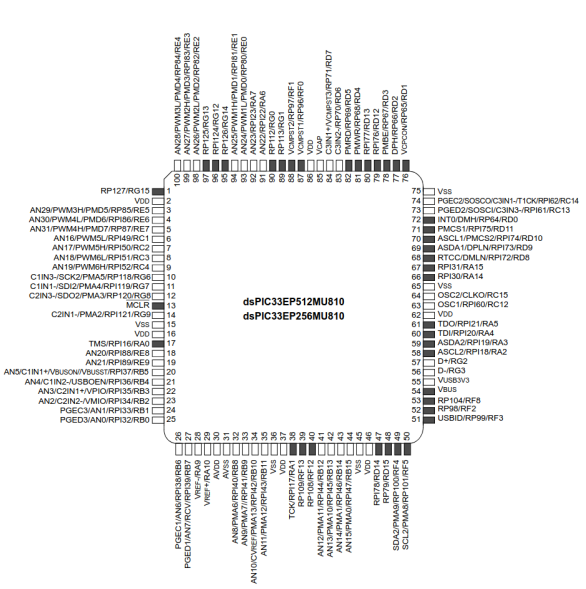


Figure 2: dsPIC33EP512MU810 pinout

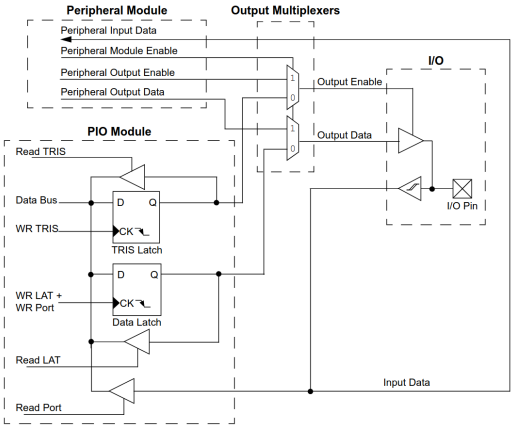


Figure 3: Typical shared port structure

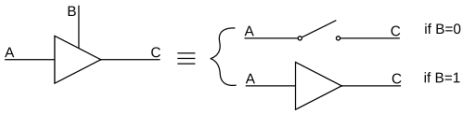


Figure 4: Three state logic

Digital IO port register

- `TRISx` is the register used to select which pins of a register are to be used as input and which one as output;
- `PORTx` is the register used to read the voltage value of the pins;
- `LATx` is register is used to write a logic value as pins output;

A read of the port latch register returns the settings of the output drivers, whilst a read of the port register returns the logic levels seen on the pins.

The `x` stands for any of the available I/O Registers: A,B,C,D,E,F. So we have TRISA register and TRISAbits bit field data structure, and so on.

Multiplexed I/O pins

Modern microcontrollers possess a vast array of internal hardware peripherals (like UART, SPI, ADC, or Timers) but are physically limited by the number of external pins on their package. To solve this bottleneck, Multiplexed I/O allows a single physical pin to serve multiple, selectable functions.

Internally, the physical pin is connected to an electronic multiplexer (MUX). Based on the values written to specific configuration registers, this MUX routes the pin's electrical signal either to the standard Digital I/O logic (controlled by `TRIS`, `PORT`, and `LAT`) or directly to an alternative peripheral module.

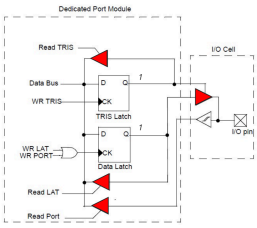
Why it is used: It drastically reduces the chip's package size and manufacturing cost while giving designers the flexibility to tailor the microcontroller to the specific needs of their application.

Before using a multiplexed pin as a standard Digital I/O, you must explicitly disable its alternate functions via software. Crucially, **pins with analog capabilities default to analog inputs** upon reset to prevent excess current draw from floating signals. To use them digitally, you must clear the corresponding bit in the `ANSELx` (Analog Select) register.

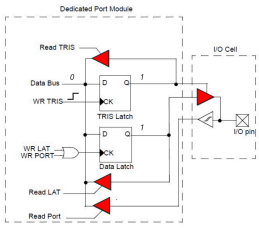
For advanced digital routing, dsPIC devices also utilize **Peripheral Pin Select (PPS)** registers, which allow you to dynamically map specific digital peripherals to designated "remappable" pins at runtime.

PinMode write

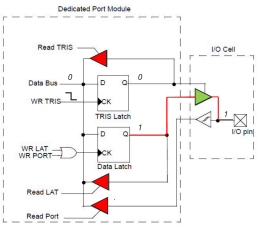
Objective: output the logic value stored in the Data Latch



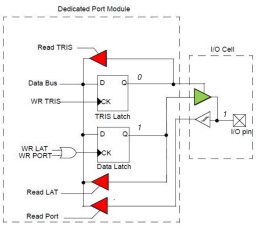
Base state



WR TRIS = 0



WR TRIS = 1

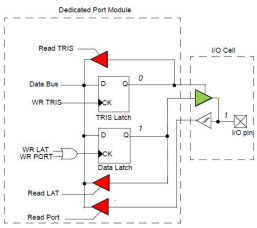


Final state

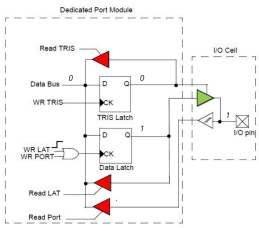
When the instruction is done, the DATA Latch (memory element) keeps the pin to 0

Data Latch change (write)

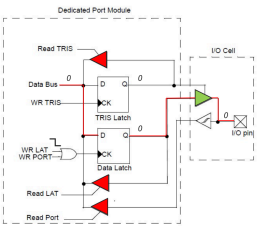
Objective: while PinMode is set to write, change the value stored in the Data Latch with the one in Data Bus



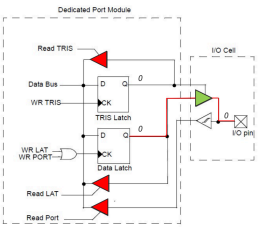
Initial state



WR LAT = 0



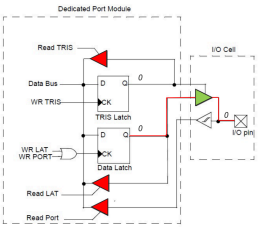
WR LAT = 1



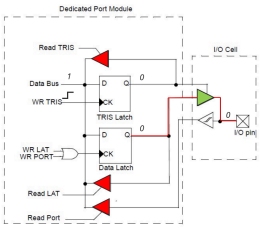
Final state

Switch to PinMode read

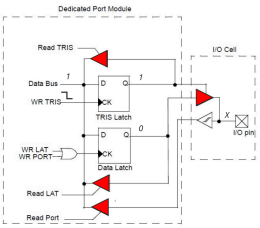
Objective: disable the writing-mode



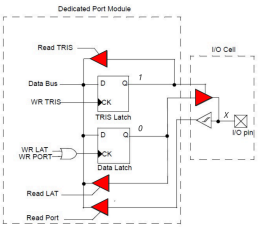
Initial state



WR TRIS = 0



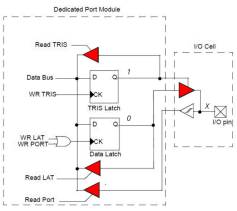
WR TRIS = 1



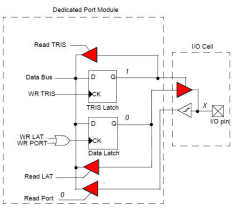
Final state

DigitalRead

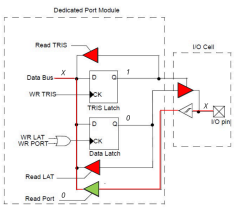
Objective: write the logic value of the pin in the Data Bus (Input buffer)



Initial state



Read Port = 0



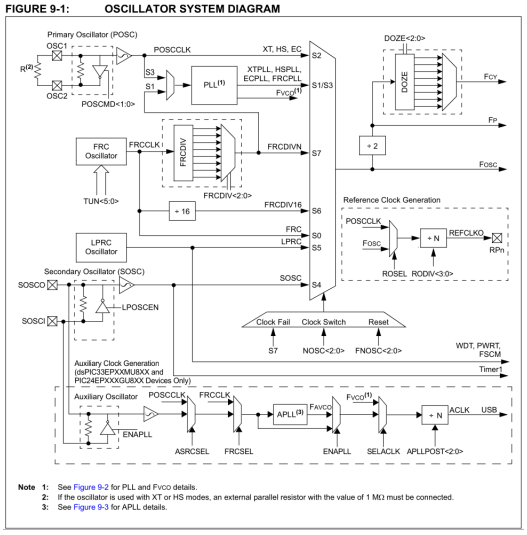
Data bus

Lesson 3.1: Oscillator

An electronic oscillator is an electronic circuit that converts direct current energy into a periodic, alternating current signal. The most common variation is the crystal oscillator, in which the output frequency is governed by a piezoelectric resonator, typically a vibrating quartz crystal. In digital electronics, this circuit is widely used to provide a precise periodic signal to a microcontroller to act as its system clock.

Oscillator Circuit for dsPIC33

FIGURE 9-1: OSCILLATOR SYSTEM DIAGRAM



Note 1: See Figure 9-2 for PLL and FVCO details.
2: If the oscillator is used with XT or HS modes, an external parallel resistor with the value of 1 MΩ must be connected.
3: See Figure 9-3 for APLL details.

TABLE 9-1: CONFIGURATION BIT VALUES FOR CLOCK SELECTION

Oscillator Mode	Oscillator Source	POSCMD<1:0>	FNOSC<2:0>	See Notes
Fast RC Oscillator with Divide-by-N (FRCDIVN)	Internal	xx	111	1, 2
Fast RC Oscillator with Divide-by-16 (FRCDIV16)	Internal	xx	110	1
Low-Power RC Oscillator (LPRC)	Internal	xx	101	1
Secondary (Timer1) Oscillator (SOSC)	Secondary	xx	100	1
Primary Oscillator (HS) with PLL (HSPLL)	Primary	10	011	—
Primary Oscillator (XT) with PLL (XTPLL)	Primary	01	011	—
Primary Oscillator (EC) with PLL (ECPLL)	Primary	00	011	1
Primary Oscillator (HS)	Primary	10	010	—
Primary Oscillator (XT)	Primary	01	010	—
Primary Oscillator (EC)	Primary	00	010	1
Fast RC Oscillator (FRC) with Divide-by-N and PLL (FRCPLL)	Internal	xx	001	1
Fast RC Oscillator (FRC)	Internal	xx	000	1

Note 1: OSC2 pin function is determined by the OSCIOFNC Configuration bit.
2: This is the default oscillator mode for an unprogrammed (erased) device.

> ***[Note]*** MikroE `clicker2` for dsPIC33 board has an XT oscillator at 8MHz, we will use this configuration with PLL (Oscillator Source: S3)

Clock System Architecture:

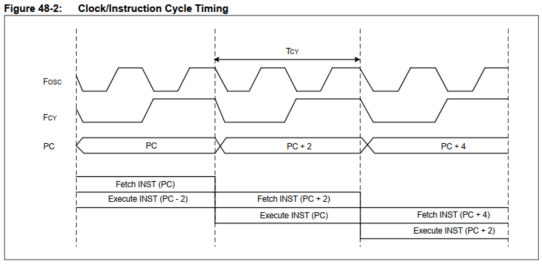
- **Multiplexed Clock Sources:** Four distinct time bases to drive the system clock (**`FOSC`**):
  - **`POSC`** (Primary Oscillator): For external high-frequency crystals, resonators, or external clock inputs.
  - **`SOSC`** (Secondary Oscillator): For low-power, low-frequency external crystals (typ. 32.768 kHz for RTC functionality).
  - **`FRC`** (Fast RC Oscillator): Internal high-frequency oscillator for fast-startup time base without external components.
  - **`LPRC`** (Low-Power RC Oscillator): Internal ultra-low-power, low-frequency oscillator, typically utilized for Deep Sleep modes and Watchdog Timer (WDT) operation.
- **`PLL`** (On-Chip Phase-Locked Loop): Integrated PLL circuitry permits frequency multiplication of the primary or internal fast oscillators to achieve maximum instruction execution rates.
- **Dynamic Clock Switching:** Hardware-managed multiplexing that allows on-the-fly transitions between clock sources during runtime, enabling firmware to dynamically scale performance and power consumption.
- **`FSCM`** (Fail-Safe Clock Monitor): Dedicated hardware fault-detection circuit that continuously monitors the primary clock edge. Upon detecting a signal loss or oscillator failure, the FSCM triggers a non-maskable interrupt (NMI) and automatically executes a failover to the internal FRC to ensure continuous, safe system operation.

F_{OSC} and F_{CY}

In microcontroller architecture, the system clock is derived from a primary oscillator (F_{OSC}). However, the central processing unit (CPU) and synchronous internal peripherals operate on the instruction cycle frequency (F_{CY}). F_{CY} represents the execution rate of a single, single-cycle machine instruction.

The relationship between F_{OSC} and F_{CY} is strictly architecture-dependent and must be explicitly verified within the specific device's datasheet. For example, in the Microchip dsPIC33 architecture, the instruction clock is derived as follows:

Figure 48-2: Clock/Instruction Cycle Timing



EQUATION 9-1: DEVICE OPERATING FREQUENCY

$$FCY = FOSC/2$$

Note: Other architectures may employ different prescalers or utilize Phase-Locked Loops (PLLs) to synthesize higher internal clock frequencies

Critical Role of F_{CY} in System Configuration

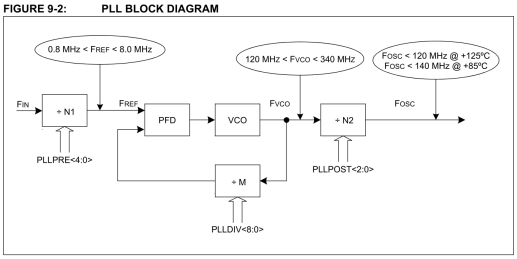
Accurate determination of F_{CY} is a prerequisite for firmware development, as it functions as the absolute temporal reference for the microcontroller. Miscalculating F_{CY} propagates systemic timing errors across all time-dependent and frequency-dependent subsystems.

F_{CY} governs the quantitative configuration of the following components:

- **Timers and Counters:** Establishes the fundamental increment rate before prescaling.
- **Communication Interfaces (UART, SPI, I2C, CAN):** Serves as the dividend in equations calculating Baud Rate Generator (BRG) register values.
- **Pulse Width Modulation (PWM):** Determines both the maximum switching frequency and the duty cycle resolution.
- **Deterministic Software Delays:** Required to compute exact cycle counts for blocking delay functions.

F_{OSC} and F_{IN}

FIGURE 9-2: PLL BLOCK DIAGRAM



EQUATION 9-2: FOSC CALCULATION

$$FOSC = FIN \times \left(\frac{M}{N1 \times N2}\right) = FIN \times \left(\frac{(PLLDIV+2)}{(PLLPRE+2) \times 2(PLLPOST+1)}\right)$$

Where,

$N1 = PLLPRE + 2$

$N2 = 2 \times (PLLPOST + 1)$

$M = PLLDIV + 2$

EQUATION 9-3: FVCO CALCULATION

$$FVCO = FIN \times \left(\frac{M}{N1}\right) = FIN \times \left(\frac{(PLLDIV+2)}{(PLLPRE+2)}\right)$$

[!] F_{IN} is the frequency of the source before entering the PLL (8MHz for the XT source)

Acronym	Component Name	Technical Description & Operational Mechanism
PFD	Phase Frequency Detector	Acts as the closed-loop error comparator. It continuously measures the phase and frequency differential between the input reference clock (F_{REF}) and the divided-down feedback clock from the VCO. It generates discrete "up" or "down" pulses proportional to this error, which a loop filter integrates into an analog DC control voltage.
VCO	Voltage-Controlled Oscillator	A high-frequency analog signal generator. Its instantaneous output frequency (F_{VCO}) is strictly dictated by the incoming DC control voltage from the PFD. It actively adjusts its oscillation rate to drive the PFD's measured error to zero, operating within strict hardware-defined frequency bounds (e.g., 100 MHz $< F_{VCO} <$ 200 MHz).
LOCK	Phase-Locked State	The stable equilibrium condition of the PLL. This state is achieved when the PFD detects zero phase and frequency error between the reference and feedback clocks. The control voltage stabilizes, forcing the VCO to maintain a mathematically precise, constant high-frequency output.

[!] Only for this course, the configuration is set by the booloader and cannot be changed. It is the following one

```
PLLFBD = 70; // M = 72
CLKDIVbits.PLLPOST = 0x00; // N1 = 2
CLKDIVbits.PLLPRE = 0x00;  // N2 = 2
```

thus, `F_CY = 72MHz`

Lesson 3.2: Timers

9 16-bit timers available:

T1: **Type A**	T2, T4, T6, T8: **Type B**	T3, T5, T7, T9: **Type C**
• can be operated also by the LPRC • can work in asynchronous counter mode	• can be concatenated with C-type leading to a 32-bit timer	• T3 and T5 can trigger an A/D conversion

Functions performed:

ST	Synchronous Timer	increments at every internal clock cycle (A,B,C)
SC	Synchronous Counter	increments at every external clock cycle (A,B,C)
AC	Asynchronous Counter	increments at every external raising signal (A)
GT	Gated Timer	counts the number of clock cycles an external signal is "high" (A,B,C)

Counter and Gated Mode

Counter Mode In Counter Mode, the timer module increments its counter register (`TMRx`) based on edge transitions detected on a dedicated external input pin (typically designated as `TxCK`), rather than the internal instruction cycle clock (F_{CY}). This mode is utilized to count discrete external events.

- **Synchronous Counter:** The external signal is synchronized with the internal system clock before incrementing the timer. The external frequency must comply with the limit relative to F_{CY}.
- **Asynchronous Counter (Type A / Timer 1 only):** The timer increments on the external signal entirely independent of the internal system clock. This architectural feature allows Timer 1 to continue functioning and generating interrupts even when the CPU and main oscillator are disabled in Sleep mode (e.g., functioning as a Real-Time Clock using a 32.768 kHz external crystal).

Gated Timer Mode (Time Accumulation) In Gated Time Accumulation Mode, the timer utilizes the internal instruction clock (F_{CY}) as its time base, but the counting operation is strictly governed by the logic state of an external pin.

- The `TMRx` register accumulates internal clock pulses exclusively while the designated external gate signal is asserted (typically driven to a logic "High").
- Upon the external signal transitioning to a de-asserted state (logic "Low"), the timer halts. The accumulated integer value residing in `TMRx` provides a highly accurate, cycle-exact measurement of the external signal's temporal duration (pulse width). This mode is essential for measuring external pulse durations, such as ultrasonic sensor echo returns or decoding external PWM inputs.

> ***[Note]*** Use case: This is a hardware feature used to measure exactly how long an external pulse lasts (e.g., measuring the echo pulse from an ultrasonic distance sensor) without needing to write complex interrupt code.

> **[!] 16-bit Hardware Limitation**
> The mathematical requirement that the calculated timer steps must be less than or equal to 65,535 is a strict physical constraint of the microcontroller's architecture.
>
> The Period Register (`PRx`) and Timer Counter (`TMRx`) are physically implemented as 16-bit memory registers. The absolute maximum integer value a 16-bit register can hold is 65,535.
>
> Attempting to load a computed value larger than this threshold (such as 400,000) will result in silent integer truncation—the hardware will discard the most significant bits and only store the remainder. Consequently, your timer will overflow at an arbitrary, drastically shorter interval, entirely destabilizing the system's timing.
>
> If a required time delay yields a step count greater than 65,535, you can:
> - mathematically reduce it by increasing the prescaler (`TCKPS`)
> - concatenate two 16-bit timers into a single 32-bit timer.
> - reuse the same timer multiple times anc count the number of completed cycles of it

Timer registers

Register 12-1: TxCON: Type A Time Base Register

Upper Byte:							
R/W-0	U-0	R/W-0	U-0	U-0	U-0	U-0	U-0
TON	—	TSIDL	—	—	—	—	—
bit 15							bit 8

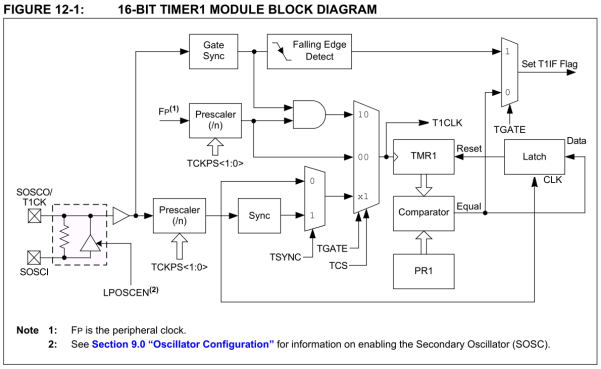
Lower Byte:							
U-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0	R/W-0	U-0
—	TGATE	TCKPS<1:0>		—	TSYNC	TCS	—
bit 7							bit 0

The configuration, scaling, and runtime execution of the hardware timers are governed by three primary Special Function Registers (SFRs) per timer module:

- **`TxCON`** (Timer x Control Register): The primary control register that dictates the operational mode, clock source, and prescaler division. Critical bitfields include:
  - **`TON`** (Timer On): Enables (1) or disables (0) the timer module. To prevent false triggers, this must be the final bit asserted during the initialization sequence.
  - **`TCS`** (Timer Clock Source Select): Multiplexes the input time base between the internal instruction cycle clock (F_{CY}) (0) or an external transition on the dedicated `TxCK` pin (1).
  - **`TGATE`** (Timer Gated Time Accumulation Enable): Enables (1) or disables (0) the gated counting mode when `TCS` is set to the internal clock. This bit turns the timer into a pulse-width measurer. It allows an external signal to act as a physical "pause/play" button for the timer.
    - **`TGATE = 0`**: The timer counts continuously as long as it is turned on (TON = 1). It doesn't care about what is happening on the outside world.
    - **`TGATE = 1`**: The timer is hooked up to an external pin (TxCK). Even if the timer is turned on (TON = 1), it will only count while that external pin is receiving a HIGH voltage signal. The moment the external signal drops to LOW, the timer pauses.
  - **`TCKPS<1:0>`** (Timer Input Clock Prescale Select): Configures the integer prescaler ratio (typically 1:1, 1:8, 1:64, or 1:256) applied to the selected clock source before it increments the core counter, allowing the timer to span longer durations without overflowing.
- **`PRx`** (Period Register): A 16-bit holding register containing the target temporal threshold. While the timer operates, the hardware continuously compares the `TMRx` value against `PRx`. Upon a strict mathematical match, the hardware automatically resets `TMRx` to zero on the subsequent clock cycle and asserts the Timer Interrupt Flag (`TxIF`, accessible at `IFSxbits.TxIF`).
- **`TMRx`** (Timer Register): The actual 16-bit counter variable that increments according to the prescaled clock source. Firmware can synchronously read from or write to this register to poll the elapsed time or force a specific phase offset.

Note on 32-bit Concatenation: When a Type B timer (even-numbered, e.g., Timer 2) is concatenated with a Type C timer (odd-numbered, e.g., Timer 3) to synthesize a 32-bit timer, the Type B module provides the Least Significant Word (LSW) and the Type C module provides the Most Significant Word (MSW). Consequently, the core registers physically combine into `TMR3:TMR2` and `PR3:PR2`. During this mode, the configuration and control logic are entirely managed via the Type B control register (`T2CON`).

FIGURE 12-1: 16-BIT TIMER1 MODULE BLOCK DIAGRAM



Note 1: FP is the peripheral clock.
2: See **Section 9.0 "Oscillator Configuration"** for information on enabling the Secondary Oscillator (SOSC).

Before setup:

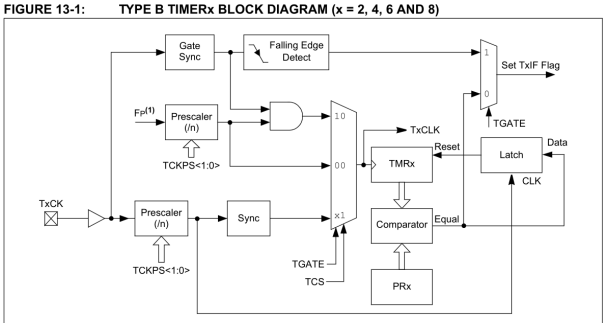
```
TxCON = 0x0000; // Disable timer and clear configs
TMRx = 0;       // Clear the counter register
```

ST	TCS = 0, TGATE = 0
SC	TCS = 1, TGATE = 0, TSYNC = 1
AC	TCS = 1, TGATE = 0, TSYNC = 0 **(A only)**
GT	TCS = 0, TGATE = 1

After setup:

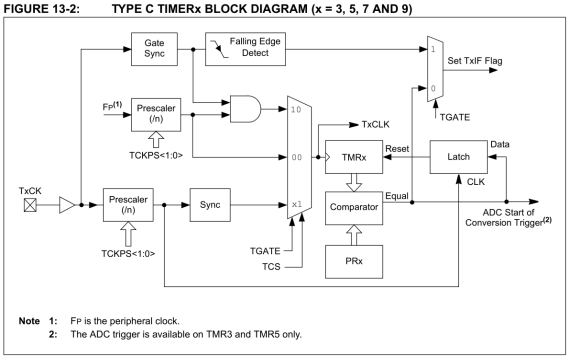
```
TxCONbits.TON = 1; // Start the timer
```

FIGURE 13-1: TYPE B TIMERx BLOCK DIAGRAM (x = 2, 4, 6 AND 8)



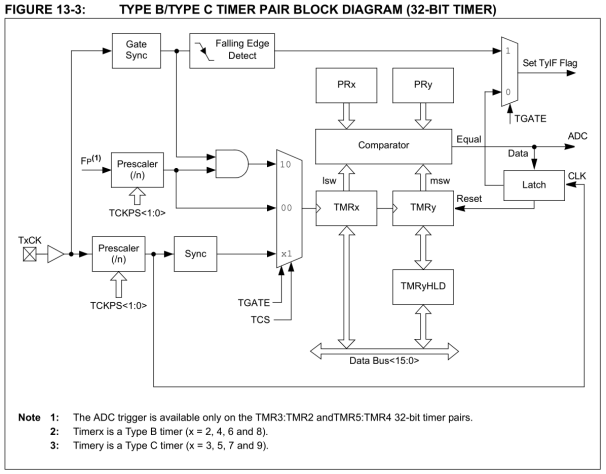
Note 1: FP is the peripheral clock.

FIGURE 13-2: TYPE C TIMERx BLOCK DIAGRAM (x = 3, 5, 7 AND 9)



Note 1: FP is the peripheral clock.
2: The ADC trigger is available on TMR3 and TMR5 only.

FIGURE 13-3: TYPE B/TYPE C TIMER PAIR BLOCK DIAGRAM (32-BIT TIMER)



Note 1: The ADC trigger is available only on the TMR3:TMR2 andTMR5:TMR4 32-bit timer pairs.
2: Timerx is a Type B timer (x = 2, 4, 6 and 8).
3: Timery is a Type C timer (x = 3, 5, 7 and 9).

Enable 32-bit timer (e.g., T2 + T3):

```
// Set 32-bit mode in Type B control reg
T2CONbits.T32 = 1;

// Example of combined Period Register setup
PR3 = 0xDEAD;      // Most Significant Word (MSW)
PR2 = 0xBEEF;      // Least Significant Word (LSW)
```

Lesson 4.1: Interrupts

Interrupt: an asynchronous signal from hardware indicating the need for attention (hardware interrupt) or a synchronous event in software indicating the need for a change in execution (software interrupt or software trap)

- **Synchronous** means that the interrupt depends on the instruction we are currently executing (e.g. a divide by zero trap)
- **Asynchronous** means that it does have any temporal relation to which part of the code the microcontroller is executing (the timer expires independently of which point the program has currently reached)

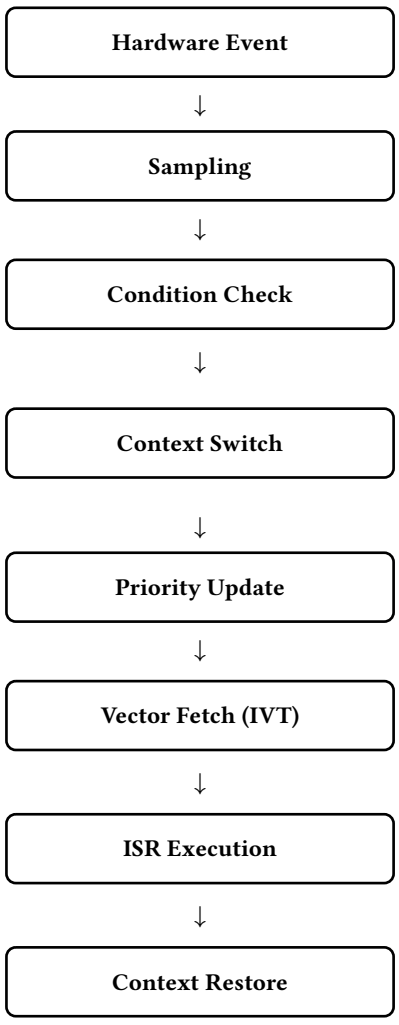
ISR (Interrupt Service Routine)

An ISR is like any other C function with some special rules:

- Each ISR has a special name which is used for linking its code to the IVT
- No parameters can be passed to the ISR
- No parameters can be returned by an ISR (void return type is mandatory)
- the flag generating the interrupt should be generally cleared in the ISR
- ISR code should not contain blocking sections
- ISRs should not be called by main line code
- Care must be taken with shared data

```
void __attribute__((__interrupt__, __auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  [...] // do some action
}
```

Interrupts: How They Work



1. Hardware Event
A special event occurs and the peripheral hardware sets its interrupt flag(s).

2. Sampling
Interrupt flags are sampled at the beginning of each instruction cycle.

3. Condition Check
IF (request is pending) **AND** (interrupt generation is enabled):
An interrupt is presented to the processor.

4. Context Switch
The processor saves the current state to the software stack:
- Current Program Counter (PC) value
- Processor Status register

5. Priority Update
The CPU sets its priority level equal to that of the pending interrupt.

6. Vector Fetch (IVT)
The CPU uses the specific interrupt source to look up the correct “slot” in the Interrupt Vector Table, retrieving the exact memory address of the ISR.

7. ISR Execution
The CPU jumps to the address provided by the IVT and executes your Interrupt Service Routine (ISR) code.

8. Context Restore
The previous context is recovered from the stack, and the main program execution continues.

The Interrupt Vector Table (IVT) and ISR Mapping

When a hardware interrupt occurs, the CPU needs to know exactly **where** in the program memory the corresponding code (the Interrupt Service Routine) is located. This linkage is strictly managed by the **Interrupt Vector Table (IVT)** and the compiler.

- **The IVT:** A dedicated, fixed array in the microcontroller's lowest program memory. It acts like a directory. Each peripheral's interrupt source has a hardcoded, pre-assigned "slot" (or Vector) in this table.
- **The Content:** Each slot in the IVT doesn't contain code; it contains a `GOTO` instruction pointing to the actual memory address where your ISR function lives.
- **The ISR Name:** To tell the compiler which IVT slot to populate, you must use the exact **ISR Name** specified in the microcontroller's documentation (e.g., `_T1Interrupt`).

1. Hardware Event
Timer 1 matches PR1
`T1IF` becomes 1

⟶

2. The IVT
CPU jumps to Vector 11
(Points to `0x1A00`)

⟶

3. Program Memory (`0x1A00`)

```
void _T1Interrupt() {
  // ISR Code executes
}
```

[!] Naming Rule:
A misspell in the ISR name won't raise any flag during compilation, but it will treat it as a standard C function. The IVT slot will remain empty or point to a default error handler. When the interrupt triggers, the microcontroller will likely reset or crash

Note also that almost all XC16 interrupt names start with a single underscore (_)

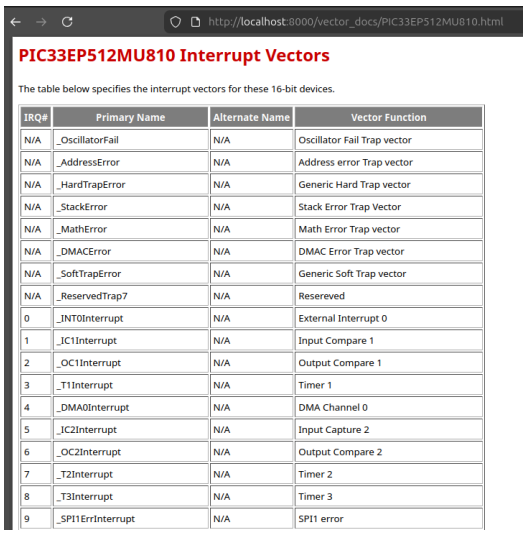
Standard ISR Code Example (XC16 Compiler):

```
void __attribute__((interrupt, no_auto_psv)) _T1Interrupt(void) {
    // 1. Perform the required urgent action
    // 2. Clear the interrupt flag, otherwise the ISR will trigger again infinitely.
    IFS0bits.T1IF = 0;
}
```

Where to find ISR Names?

The hardware datasheet describes the physical Interrupt Vector Table, but the exact C-function names are defined by the XC16 Compiler.

The list of ISR names for a specific microcontroller can be found in the XC16 installation directory:
`/opt/microchip/xc16/v2.10/docs/vector_docs`



PIC33EP512MU810 Interrupt Vectors

The table below specifies the interrupt vectors for these 16-bit devices.

IRQ#	Primary Name	Alternate Name	Vector Function
N/A	_OscillatorFail	N/A	Oscillator Fail Trap vector
N/A	_AddressError	N/A	Address error Trap vector
N/A	_HardTrapError	N/A	Generic Hard Trap vector
N/A	_StackError	N/A	Stack Error Trap Vector
N/A	_MathError	N/A	Math Error Trap vector
N/A	_DMACError	N/A	DMAC Error Trap vector
N/A	_SoftTrapError	N/A	Generic Soft Trap vector
N/A	_ReservedTrap7	N/A	Resereved
0	_INT0Interrupt	N/A	External Interrupt 0
1	_IC1Interrupt	N/A	Input Compare 1
2	_OC1Interrupt	N/A	Output Compare 1
3	_T1Interrupt	N/A	Timer 1
4	_DMA0Interrupt	N/A	DMA Channel 0
5	_IC2Interrupt	N/A	Input Capture 2
6	_OC2Interrupt	N/A	Output Compare 2
7	_T2Interrupt	N/A	Timer 2
8	_T3Interrupt	N/A	Timer 3
9	_SPI1ErrInterrupt	N/A	SPI1 error

[Note] Visualize the website in case Firefox is *sandboxed*: the Local Web Server Trick

Since Firefox is blocking a local file path (`file:///`), we can trick Firefox by turning that folder into a temporary website. Almost all Linux systems come with Python pre-installed, which has a built-in mini web server.

1. Open your Terminal.
2. Navigate to the documentation folder by pasting this command and hitting Enter:
   `cd /opt/microchip/xc16/v2.10/docs/`
3. Start the temporary server by running this command:
   `python3 -m http.server 8000`
4. Now, go back to Firefox and type this exact address into the URL bar:
   `http://localhost:8000/XC16MasterIndex.htm`

When you are done reading, just go back to the terminal and press Ctrl + C to stop the server

Common interrupts for th dsPIC33

Peripheral	Interrupt Flag (IFSx)	Required XC16 ISR Name
Timer 1	`T1IF`	`void _T1Interrupt(void)`
Timer 2	`T2IF`	`void _T2Interrupt(void)`
Timer 3	`T3IF`	`void _T3Interrupt(void)`
UART 1 RX	`U1RXIF`	`void _U1RXInterrupt(void)`
UART 1 TX	`U1TXIF`	`void _U1TXInterrupt(void)`
ADC 1	`AD1IF`	`void _ADC1Interrupt(void)`
External Int 0	`INT0IF`	`void _INT0Interrupt(void)`

Nested Interrupts

A strictly higher-priority interrupt will preempt an active Interrupt Service Routine (ISR), elevating the CPU's execution priority and creating a nested interrupt. Once the higher-priority routine finishes, the CPU resumes the preempted ISR. Conversely, equal or lower-priority interrupts cannot preempt an active routine; they are instead pended (delayed). If a pended interrupt's conditions are triggered multiple times during this delay, the events are coalesced into a single ISR execution when the current routine completes.

Interrupt Flag Status Registers

Register 6-5: IFS0: Interrupt Flag Status Register 0

Upper Byte:							
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CNIF	MI2CIF	SI2CIF	NVMIF	ADIF	U1TXIF	U1RXIF	SPI1IF
bit 15							bit 8

Lower Byte:							
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
T3IF	T2IF	OC2IF	IC2IF	T1IF	OC1IF	IC1IF	INT0IF
bit 7							bit 0

bit 15 **CNIF:** Input Change Notification Flag Status bit
1 = Interrupt request has occurred
0 = Interrupt request has not occurred

bit 14 **MI2CIF:** I^2C Bus Collision Flag Status bit
1 = Interrupt request has occurred
0 = Interrupt request has not occurred

`IFSx`: Interrupt Flag Status registers, contains the flags that notify th program of certain events. For example IFS0bits.T1IF tells if the Timer 1 has expired. These flags are set to `1` automatically, but must be set to `0` by the user.

Page 58 of the datasheet shows the location of all the IFs

Interrupt Enable Control Registers

Register 6-8: IEC0: Interrupt Enable Control Register 0

Upper Byte:							
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CNIE	MI2CIE	SI2CIE	NVMIE	ADIE	U1TXIE	U1RXIE	SPI1IE
bit 15							bit 8

Lower Byte:							
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
T3IE	T2IE	OC2IE	IC2IE	T1IE	OC1IE	IC1IE	INT0IE
bit 7							bit 0

bit 15 **CNIE:** Input Change Notification Interrupt Enable bit
1 = Interrupt request enabled
0 = Interrupt request not enabled

bit 14 **MI2CIE:** I^2C Bus Collision Interrupt Enable bit
1 = Interrupt request enabled
0 = Interrupt request not enabled

`IECx`: Interrupt Enable Control example, it is used to enable an interrupt. For example `IEC0bits.T1IE = 1` tells that we want an interrupt every time that IFS0bits.T1IF becomes `1`.

see Chapter 6, pag. 148 of the reference manual

Interrupts and Shared Variables

When the main program and an Interrupt Service Routine (ISR) need to exchange information (such as a hardware flag, a sensor reading, or a timer tick), they must use global shared variables. Because ISRs execute asynchronously, sharing data introduces specific compiler and hardware level risks.

1. The `volatile` Keyword: Breaking the Single-Thread Illusion
In standard C programming, the compiler analyzes your code and aggressively optimizes it. For example, if it sees a loop like `while(flag == 0);` and nothing **inside** that loop modifies `flag`, the compiler assumes `flag` will never change. To save time, it reads `flag` once, caches it in a CPU register, and turns your code into an inescapable infinite loop.

Embedded systems, however, are non-deterministic. Variables can change without the compiler's knowledge. The `volatile` keyword is a strict warning to the compiler: **"Do not optimize or cache this variable. Its value can change at any time, outside of the current execution flow."**

The `volatile` qualifier is mandatory in three generic embedded scenarios:
- **Global Variables Shared with ISRs:** Hardware interrupts can pause the main loop at any microsecond and alter shared data invisibly to the compiler.
- **Memory-Mapped Hardware Registers:** The value of peripheral registers (like `PORTB` or `T1IF`) changes based on physical electrical signals or peripheral hardware logic, not software instructions.
- **Multithreading/RTOS:** When multiple tasks share the same memory space in a Real-Time Operating System.

By declaring a variable as `volatile`, you force the CPU to physically fetch the value from RAM (or the hardware register) on **every single read**, guaranteeing the code always acts on the true, real-time state of the system.

```
volatile uint16_t timer_ticks = 0; // The compiler will not cache this
```

2. The Read-Modify-Write Vulnerability
Even when using `volatile`, modifying shared data is dangerous. A simple C statement like `timer_ticks++` or `timer_ticks += 5` is not executed in a single clock cycle. It is translated by the compiler into multiple assembly instructions:
1. **Read:** Fetch the current value from RAM into a working register.
2. **Modify:** Perform the ALU operation (e.g., add 1).
3. **Write:** Store the updated value back into RAM.

The Crash Scenario:
If an interrupt triggers exactly after the **Read** step but before the **Write** step, the main program is suspended holding a "stale" copy of the variable. If the ISR then modifies that same variable in RAM and returns, the main program will blindly overwrite the ISR's new data with its own stale data.

This unpredictable data corruption is what necessitates the use of specialized protections, known as **Critical Regions**.

Critical Regions

A critical section is a segment of code that accesses shared resources where preemption by an ISR could cause data corruption or race conditions. To protect these regions, interrupts are temporarily masked (disabled) during their execution. If a triggering event occurs while masked, the hardware still sets the Interrupt Flag (IF), creating a pending interrupt that executes immediately once the interrupt is re-enabled. However, because the IF is a single bit, multiple occurrences of the same event during this masked period are coalesced; the flag remains 1, and only a single ISR execution will occur upon unmasking, resulting in the loss of the intermediate events.

> ***[Note]*** A statement can also be protected from interrupt using the `INTERRUPT_PROTECT` macro.
>
> ```
> INTERRUPT_PROTECT(LATGbits.LATG15 ^= 1);
> // Not interruptible by level 1-7 interrupt requests and safe at any optimization level
> ```
>
> *Pag 175 MPLAB XC16 C Compiler User Guide*

Lesson 4.2: Peripheral Pin Select (PPS)

Here is an expanded, highly technical explanation based on Option 2. It provides the necessary architectural depth while setting up a clear transition to your next topics (input and output mapping).

Architectural Overview: Peripheral Pin Select (PPS)

Unlike microcontrollers with statically assigned pin functions, the `dsPIC33EP512MU810` utilizes an internal routing matrix: a software-configurable crossbar switch. This matrix decouples the internal digital peripherals (such as UARTs, SPI modules, Timers, and External Interrupts) from the physical General Purpose I/O (GPIO) pins.

Hardware Implications and Constraints

The primary advantage of PPS is significant PCB layout optimization. Hardware engineers can route physical traces based on spatial convenience and signal integrity rather than being constrained by a rigid, factory-defined pinout.

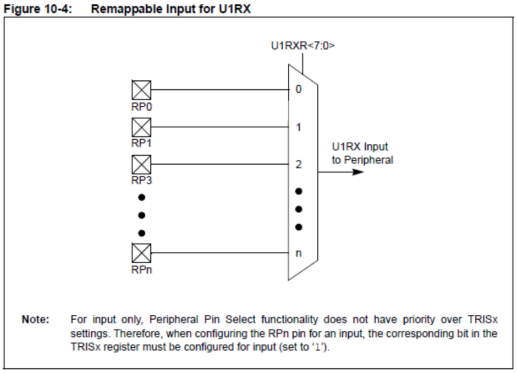
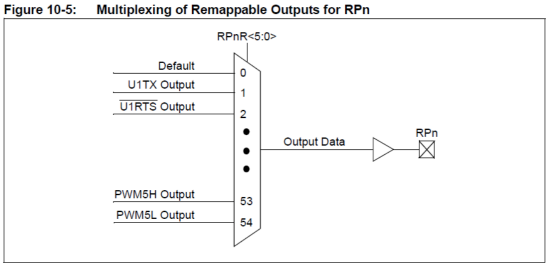
However, this architecture introduces two critical design constraints:

- **Software Dependency:** By default, remapable peripherals are disconnected from the physical pins. The physical hardware (like the button wired to **`RE8`**) will remain inert until the firmware explicitly initializes the routing matrix during boot.
- **Digital Exclusivity:** PPS multiplexing applies strictly to **digital** signals. Analog peripherals (such as ADCs or analog comparators) and specialized pins (like I2C, power, or oscillators) remain physically hardwired and cannot be remapped.

Implementation Mechanism

To bridge the gap between the physical PCB layout and the microcontroller's internal logic (e.g., assigning the trace at **`RE8`** to the **`INT1`** module), the firmware must configure dedicated PPS Special Function Registers (SFRs).

Because signal direction dictates how the internal multiplexers route the data, this configuration must be explicitly handled in two distinct phases: **Input Mapping** and **Output Mapping**.

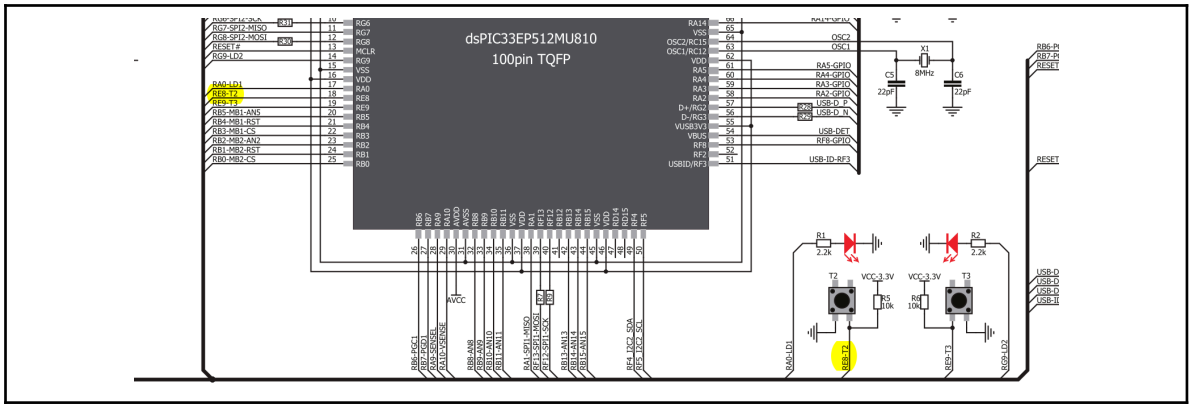
Input Mapping	Output Mapping
Figure 10-4: Remappable Input for U1RX  **Note:** For input only, Peripheral Pin Select functionality does not have priority over TRISx settings. Therefore, when configuring the RPn pin for an input, the corresponding bit in the TRISx register must be configured for input (set to '1'). `[!] Functionality = Remappable Input Pin`	**Figure 10-5: Multiplexing of Remappable Outputs for RPn**  `[!] Remappable Output Pin = Functionality (Peripheral output)`

Input mapping example: Assigning External Interrupt 1 to the Left Button

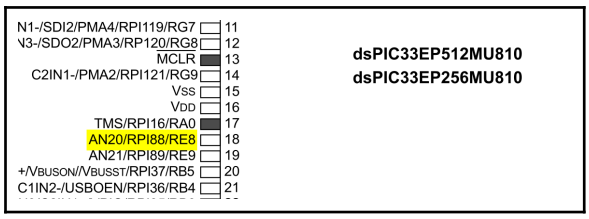
Objective: *route the physical signal from the left button (wired to pin **RE8**) to the internal module **INT1** so that pressing the button triggers an interrupt without any software polling.*

To route a physical signal to an internal module using Peripheral Pin Select (PPS), we must first gather two crucial pieces of information from our documentation: the **PPS Pin Identifier** and the **Peripheral Designator**.

1. **Determine the physical pin and its PPS identifier**
   - **Hardware Schematic:** First, we consult the development board's manual (`clicker-2-for-dspic33-manual-v100.pdf`). The circuit schematic indicates that the left button is physically hardwired to the microcontroller pin labeled **RE8** (numeric pin 18).



   - **Microcontroller Datasheet:** Next, we cross-reference pin 18 in the dsPIC33 datasheet's pinout diagram. The device multiplexes several functions onto this pin, listing it as `AN20/RPI88/RE8`.



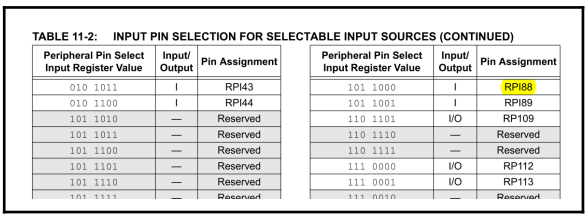
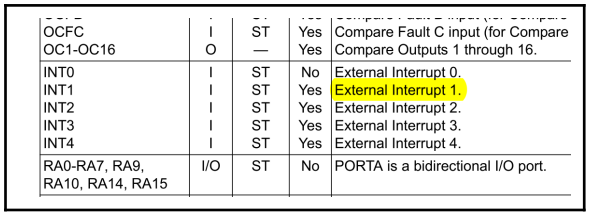


TABLE 11-2: INPUT PIN SELECTION FOR SELECTABLE INPUT SOURCES (CONTINUED)

Peripheral Pin Select Input Register Value	Input/ Output	Pin Assignment	Peripheral Pin Select Input Register Value	Input/ Output	Pin Assignment
010 1011	I	RPI43	101 1000	I	RPI88
010 1100	I	RPI44	101 1001	I	RPI89
101 1010	—	Reserved	110 1101	I/O	RP109
101 1011	—	Reserved	110 1110	—	Reserved
101 1100	—	Reserved	110 1111	—	Reserved
101 1101	—	Reserved	111 0000	I/O	RP112
101 1110	—	Reserved	111 0001	I/O	RP113

   Because we are mapping an **input** signal, we will refer to this pin in our software configuration by its Remappable Pin Input identifier: **RPI88**.

2. **Determine the internal peripheral designator**
   - Now we need the official mnemonic for the internal module we want to trigger.
   - Inside the dsPIC33 datasheet, navigate to **TABLE 1-1: PINOUT I/O DESCRIPTIONS**. Searching for "External Interrupt 1" reveals that the manufacturer designates this peripheral as **INT1**.



OCFC	I	ST	Yes	Compare Fault C input (for Compare
OC1-OC16	O	—	Yes	Compare Outputs 1 through 16.
INT0	I	ST	No	External Interrupt 0.
INT1	I	ST	Yes	External Interrupt 1.
INT2	I	ST	Yes	External Interrupt 2.
INT3	I	ST	Yes	External Interrupt 3.
INT4	I	ST	Yes	External Interrupt 4.
RA0-RA7, RA9, RA10, RA14, RA15	I/O	ST	No	PORTA is a bidirectional I/O port.

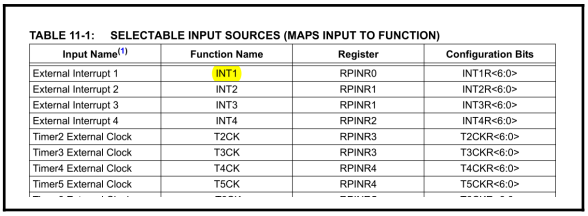


TABLE 11-1: SELECTABLE INPUT SOURCES (MAPS INPUT TO FUNCTION)

Input Name[1]	Function Name	Register	Configuration Bits
External Interrupt 1	INT1	RPINR0	INT1R<6:0>
External Interrupt 2	INT2	RPINR1	INT2R<6:0>
External Interrupt 3	INT3	RPINR1	INT3R<6:0>
External Interrupt 4	INT4	RPINR2	INT4R<6:0>
Timer2 External Clock	T2CK	RPINR3	T2CKR<6:0>
Timer3 External Clock	T3CK	RPINR3	T3CKR<6:0>
Timer4 External Clock	T4CK	RPINR4	T4CKR<6:0>
Timer5 External Clock	T5CK	RPINR4	T5CKR<6:0>

3. **Finalize the configuration**

> `[!]` `Functionality = Remappable Input Pin`

With the source (`RPI88`) and the destination (`INT1`) identified, we now have the required parameters to configure the PPS Input Mapping registers in our C code.

```
RPINR0bits.INT1R = 0b1011000;
IFS1bits.INT1IF = 0; // clear interrupt flag
IEC1bits.INT1IE = 1; // enable interrupt
```

Output mapping example:

Objective: *remap the output of Timer 1 to the physical pin* ***`RA0`*** *(numeric pin 16) to create a blinking LED effect without using any software toggling.*

1. **Get the PPS Output Identifier**: *(same procedure as previous case)* `RPI16`

> **[!] ATTENTION:**
> If you look closely at the dsPIC33 datasheet, `RA0` is labeled as `RPI16`, not `RP16`.
> - **`RP` (Remappable Pin):** Can be used for Input OR Output mapping.
> - **`RPI` (Remappable Pin Input):** Physically lacks the internal multiplexing circuitry for outputs. It can be used for Input ONLY.
>
> Because `RA0` is strictly an `RPI` pin, **it is physically impossible to route a peripheral output to it via PPS.** The compiler will not find an `RPOR` (Remappable Pin Output Register) for it because it doesn't exist in the silicon!
>
> *To make this hardware-blink example work, we must choose a pin that supports output mapping. For this exercise, let's assume we are remapping to `RD6`, which has the identifier `RP70`.*

1. **Understand Timer Outputs vs. Output Compare**
   There is a second architectural rule to remember: standard Timers (like Timer 1) do not have direct physical output pins that can be routed via the routing matrix. To blink an LED entirely in hardware, we must use an **Output Compare (OC)** module. The OC module can be configured to toggle an output automatically every time a designated timer (usually Timer 2 or Timer 3) matches a specific period.

2. **Determine the Output Function Number**
   - We need the specific numeric code for the peripheral we want to output.
   - Inside the dsPIC33 datasheet, navigate to **TABLE 4-37: PERIPHERAL PIN SELECT OUTPUT REGISTER MAP**.
   - Searching for "Output Compare 1" (`OC1`) reveals its 6-bit output value is `0b010000` (`0x10`).

TABLE 4-37: PERIPHERAL PIN SELECT OUTPUT REGISTER DEVICES ONLY

File Name	Addr.	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit
RPOR0	0680	—	—	RP65R<5:0>				
RPOR1	0682	—	—	RP67R<5:0>				
RPOR2	0684	—	—	RP69R<5:0>				
RPOR3	0686	—	—	RP71R<5:0>				
RPOR4	0688	—	—	RP80R<5:0>				
RPOR5	068A	—	—	RP84R<5:0>				
RPOR6	068C	—	—	RP87R<5:0>				
RPOR7	068E	—	—	RP97R<5:0>				
RPOR8	0690	—	—	RP99R<5:0>				
RPOR9	0692	—	—	RP101R<5:0>				

TABLE 11-3: OUTPUT SELECTION FOR REMAPPABLE PINS (RPn)

Function	RPnR<5:0>	Output Name
DEFAULT PORT	000000	RPn tied to Default Pin
U1TX	000001	RPn tied to UART1 Transmit
U1RTS	000010	RPn tied to UART1 Ready-to-Send
U2TX	000011	RPn tied to UART2 Transmit
U2RTS	000100	RPn tied to UART2 Ready-to-Send
SDO1	000101	RPn tied to SPI1 Data Output
SCK1	000110	RPn tied to SPI1 Clock Output
SS1	000111	RPn tied to SPI1 Slave Select
SS2	001010	RPn tied to SPI2 Slave Select
CSDO	001011	RPn tied to DCI Data Output
CSCK	001100	RPn tied to DCI Clock Output
COFS	001101	RPn tied to DCI FSYNC Output
C1TX	001110	RPn tied to CAN1 Transmit
C2TX	001111	RPn tied to CAN2 Transmit
OC1	010000	RPn tied to Output Compare 1 Output
OC2	010001	RPn tied to Output Compare 2 Output
OC3	010010	RPn tied to Output Compare 3 Output
OC4	010011	RPn tied to Output Compare 4 Output

1. **Finalize the configuration**

> `[!] Remappable Output Pin = Functionality (Peripheral output)`

```
// 1. Output Mapping: Route Output Compare 1 (0x10) to physical pin RP80 (RE0)
RPOR4bits.RP80R = 0x10;

// 2. Peripheral Setup: Configure Timer 2 and OC1 to blink the LED
// PR2 = ...                 // Set Timer 2 period
OC1CON1bits.OCTSEL = 0b000; // Select Timer 2 as the clock source for OC1
OC1CON2bits.SYNCSEL = 0x1F; // Free-running mode (No hardware sync required)
OC1CON1bits.OCM = 0b011;    // Configure OC1 in Toggle mode (hardware blink)
T2CONbits.TON = 1;          // Start Timer 2
```

Debouncing pattern: Interrupts and PPS

c

```c
#include <xc.h>
#include "timer.h"

void __attribute__((interrupt)) _INT1Interrupt(){
    IFS1bits.INT1IF = 0;  // Clear INT1 flag
    IEC0bits.T2IE = 1;    // Enable T2Interrupt
    tmr_setup_period(TIMER2, 10);
}

void __attribute__((interrupt)) _T2Interrupt(){
    IFS0bits.T2IF = 0;    // Clear Timer 2 flag
    IEC0bits.T2IE = 0;    // Disable Timer 2 interrupt
    T2CONbits.TON = 0;    // Turn off Timer 2

    if (PORTEbits.RE8 == 1) {
        LATAbits.LATA0 ^= 1;
    }
}

int main() {
    ANSELA = ANSELB = ANSELC = ANSELD = ANSELE = ANSELG = 0x0000;

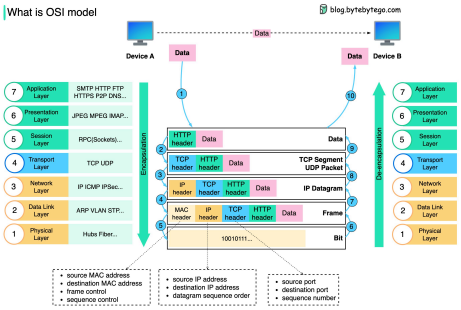
    TRISAbits.TRISA0 = 0;   // set the pin RA0 as output
    TRISEbits.TRISE8 = 1;   // set the pin RE8 as input

    RPINR0bits.INT1R = 0b1011000;
    IFS1bits.INT1IF = 0; // clear interrupt flag
    IEC1bits.INT1IE = 1; // enable interrupt

    while(1){}
}
```

Lesson 5.1: Introduction to Communication Protocols

The OSI Reference Model



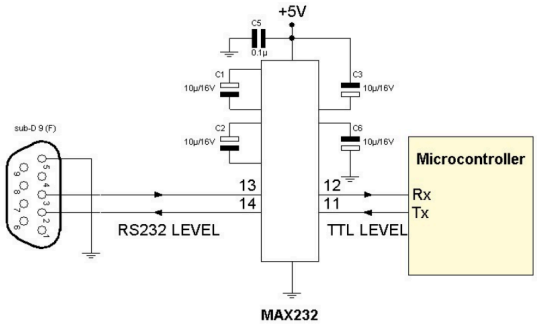
The Open Systems Interconnection (OSI) model standardizes the communication functions of a telecommunication or computing system into seven distinct abstraction layers.

In the context of microcontrollers and hardware-level digital communication, our focus is predominantly on the **Physical Layer** and the **Data Link Layer**.

- Layer 1 dictates the electrical and physical specifications of the data connection (e.g., voltage levels, transmission mediums, line impedance, and bit-level timing).
- Layer 2 handles node-to-node data transfer, framing, and basic error detection.

Line drivers

Line drivers are specialized amplifiers used to improve the signal integrity and driving capability of an analog or digital signal at its source. They act as impedance matching buffers, driving the transmission line with a higher-than-normal current or voltage level to overcome the parasitic capacitance and signal attenuation inherent to long cables.

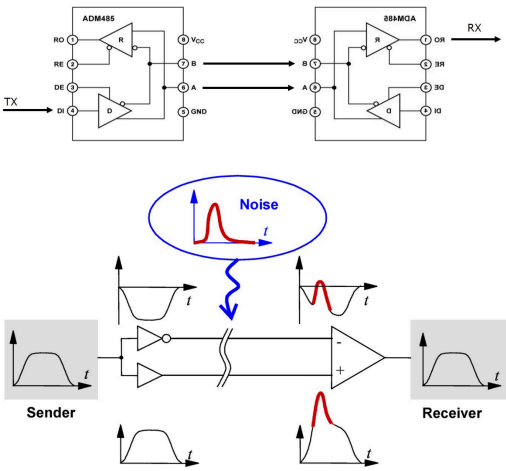


In this example, the `MAX232` line driver acts as a transceiver to convert signals from single-ended TTL logic levels (0 to 5V) to the robust RS-232 standard levels (-9V to +9V), significantly increasing noise immunity over longer cable runs.

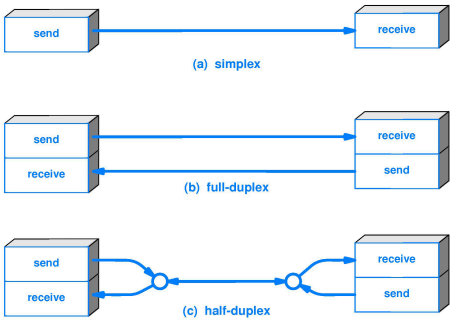
Differential signaling

Differential signaling transmits information using two complementary electrical signals sent on paired wires. The receiving circuit (a differential amplifier) extracts the data by measuring the voltage difference between the inverting and non-inverting inputs, rather than referencing a single wire to absolute ground.

This technique is highly robust and best suited for high-speed protocols like RS-485, CAN bus, and USB.



Simplex - Half-Duplex - Duplex communication



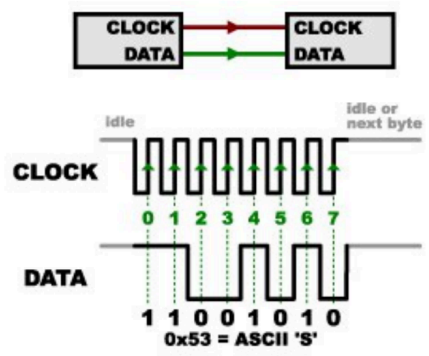
(a) A **simplex** communication system transmits data in only one direction. No return communication is possible on that channel (e.g., basic sensor telemetry or standard radio broadcasts).

(b) A **full-duplex** system allows bidirectional communication to occur simultaneously, typically requiring separate transmission and reception channels.

(c) A **half-duplex** system provides bidirectional communication only in one direction at a time. Devices must take turns transmitting and receiving over a shared medium, requiring a mechanism for collision avoidance and medium access control.

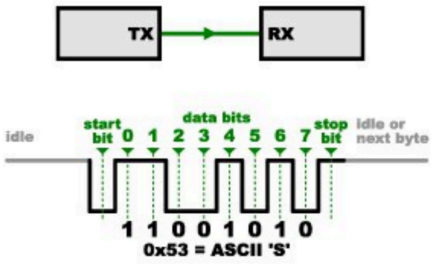
Synchronous and Asynchronous communication

Synchronous communication



In synchronous protocols (e.g., SPI, I2C), the exact moment optimal for sampling the data line is dictated by a shared, continuous external clock signal generated by the bus master. This ensures tight temporal alignment between sender and receiver, allowing for high throughput. However, this comes at the cost of routing an extra dedicated trace/cable solely for the clock, which can introduce clock skew over long distances.

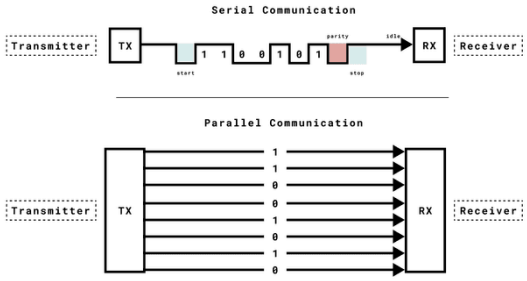
Asynchronous communication



Asynchronous communication (e.g., UART) lacks a shared clock. Instead, both devices must pre-agree on a specific data rate (Baud rate). The transmission begins with a **Start Bit** (a first transition from IDLE to LOW), which triggers an interrupt to synchronize the receiver's internal sampling clock for that specific data frame (**byte-wise sync**).

The major downside is overhead: a non-negligible portion of the bandwidth is consumed by synchronization metadata (Start, Stop, and Parity bits). For a standard 8N1 UART configuration, 20% of the transmitted bits carry no payload data.

Serial - Parallel communication



Parallel communication transmits multiple bits simultaneously over parallel data buses, offering high bandwidth. However, it is fundamentally limited over longer distances due to two main phenomena: **crosstalk** (electromagnetic interference between adjacent wires) and **clock skew** (bits arriving at slightly different times due to trace length disparities). Therefore, modern high-speed communications generally favor optimized serial transmission.

Lesson 5.2: dsPIC33 UART Architecture and Configuration

UART Transmitter and Receiver Mechanics

Data transmission and reception in the UART module rely on a dedicated clock and shift registers. The core mechanism follows this flow: `System Clock` → `Baud Rate Generator` → `Shift Register (TSR/RSR)` → `TX/RX Pin`.

A critical feature of the receiver is the **16x Oversampling**. The Baud Rate Generator (BRG) produces a clock signal that is 16 times the desired baud rate. This allows the receiver module to sample the incoming RX line multiple times per bit period, typically voting on the 7th, 8th, and 9th clock cycles to determine the bit's actual value. This dramatically increases noise immunity and tolerance for slight clock skews between the communicating devices.

Additionally, dsPIC devices typically feature a 4-level deep hardware **FIFO (First-In, First-Out)** buffer for both transmission and reception. This prevents CPU bottlenecks by allowing the program to queue multiple bytes or read bursts of incoming data without dropping frames. Interrupts can be programmed to raise a flag at specific FIFO thresholds (e.g., when the buffer is Empty, 3/4 Full, or Full).

Figure 17-3: UART Transmitter Block Diagram

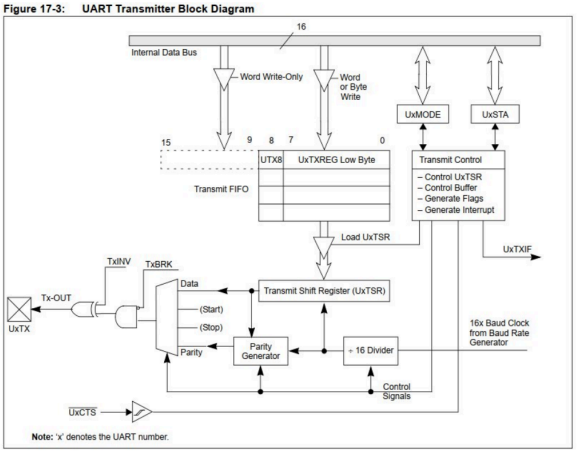
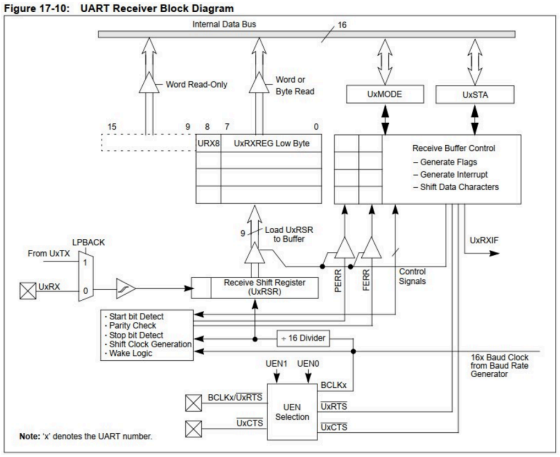


Figure 17-10: UART Receiver Block Diagram



Control and Status Registers

REGISTER 20-2: UxSTA: UARTx STATUS AND CONTROL REGISTER

R/W-0	R/W-0	R/W-0	U-0	R/W-0, HC	R/W-0	R-0	R-1
UTXISEL1	UTXINV	UTXISEL0	—	UTXBRK	UTXEN(1)	UTXBF	TRMT
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R-1	R-0	R-0	R/C-0	R-0
URXISEL<1:0>		ADDEN	RIDLE	PERR	FERR	OERR	URXDA
bit 7							bit 0

`UxSTA`: UART Status and Control Register. It contains the flags that monitor hardware events and configure buffer interactions.

- **`UTXISEL`**: Defines the condition that triggers the transmission interrupt flag.
- **`URXISEL`**: Defines the condition that triggers the reception interrupt flag.
- **`UTXEN`**: Transmit Enable bit. Must be set **after** the UART is enabled.
- **`TRMT`**: Transmit Shift Register Empty bit (Read-only). `1` means transmission is fully complete (TSR and FIFO are empty).
- **`URXDA`**: Receive Buffer Data Available bit. `1` means at least one unread character is in the RX FIFO.
- **`OERR`**: Receive Buffer Overrun Error Status bit.

Managing Overrun Errors (OERR)

An overrun occurs when the 4-level RX FIFO is full and a 5th character is completely received into the Receive Shift Register (RSR). When `OERR = 1`, the hardware strictly blocks any further incoming bytes to protect existing data.

Recovery protocol: The user must clear the error in software (`UxSTAbits.OERR = 0`). It is possible to either:

1. Read out the existing valid bytes from `UxRXREG` to rescue them, then clear `OERR`.
2. Clear `OERR` immediately (flushing the buffer and losing unread data).

Mode Registers

Register 17-1: UxMODE: UARTx Mode Register

R/W-0	U-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0	R/W-0
UARTEN	—	USIDL	IREN(1)	RTSMD	—	UEN<1:0>	
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
WAKE	LPBACK	ABAUD	URXINV	BRGH	PDSEL<1:0>		STSEL
bit 7							bit 0

`UxMODE`: UARTx Mode Register. Configures the fundamental protocol parameters.

- **`UARTEN`**: UART Enable bit. Powers up the peripheral.
- **`PDSEL`**: Parity and Data Selection bits (e.g., 8-bit no parity, 8-bit even parity, 9-bit data).
- **`STSEL`**: Stop Bit Selection (1 or 2 stop bits).
- **`BRGH`**: High Baud Rate Enable bit. Modifies the BRG multiplier (4x or 16x).

> ***[Note]* Core Data and Timing Registers**
>
> - `UxRXREG`: The memory-mapped register to read received characters (pulls from the RX FIFO).
> - `UxTXREG`: The memory-mapped register to queue characters for transmission (pushes to the TX FIFO).
> - `UxBRG`: The 16-bit Baud Rate Generator register.
>
> **Baud Rate Calculation:**
>
> $$Baud\ Rate = \frac{FCY}{16 \cdot (UxBRG + 1)}$$
>
> $$UxBRG = \frac{FCY}{16 \cdot Baud\ Rate} - 1$$
>
> **Note:** FCY denotes the instruction cycle clock frequency.

[!] For exact register mapping and advanced features like IrDA decoding or Auto-Baud, always consult the specific dsPIC33 Family Reference Manual (FRM).

Interrupt Event Selection

The timing of when the CPU is notified via an Interrupt Flag (IF) is highly configurable. This allows the software to optimize CPU load.

From the datasheet (Example for `UTXISEL`):

```
bit 15,13 UTXISEL<1:0>: UARTx Transmission Interrupt Mode Selection bits
  11 = Reserved; do not use
  10 = Interrupt when a character is transferred to the Transmit Shift Register (TSR),
       and as a result, the transmit buffer becomes completely empty.
  01 = Interrupt when the last character is shifted out of the Transmit Shift Register;
       all transmit operations are fully completed (idle bus).
  00 = Interrupt when a character is transferred to the Transmit Shift Register
       (implies there is at least one open slot in the transmit buffer).

bit 14    UTXINV: UARTx Transmit Polarity Inversion bit
  If IREN = 0 (Standard UART):
    1 = UxTX Idle state is '0' (Low)
    0 = UxTX Idle state is '1' (High - Standard default)
```

By modifying `UTXISEL` and `URXISEL`, you dictate the **condition** under which `UxTXIF` and `UxRXIF` are set to `1`. For example, setting `UTXISEL = 10` is useful if you want to refill the FIFO only when it has entirely run out of data, minimizing interrupt overhead.

> **[!] Attention:**
> `UTXISEL` and `URXISEL` dictate **when** the hardware flag is raised, but they **do not enable the interrupt itself**! To allow the CPU to jump to the Interrupt Service Routine (ISR), you must explicitly set the corresponding Enable bits (e.g., `UxTXIE` and `UxRXIE`) in the core `IECx` (Interrupt Enable Control) registers.
>
> This two-step process (setting the condition with `UTXISEL`/`URXISEL` and enabling the interrupt in `IECx`) is a common source of confusion for beginners, as it can lead to situations where the flag is correctly set by the hardware but the CPU never responds because the interrupt was not enabled.

dsPIC33 UART echo implementation

Task

```
/* TASK: Configure the UART1 to use the MikroBUS2.
 *  [ ] First find out the remappable I/O pins corresponding to the RD11
 *      and RD0 registers, then perform the remapping of UART1.
 *
 * input mapping
 *    - RD11 -> RPI75 -> 0b100 1011
 *    - UART1 receive -> RPINR18.U1RXR
 *
 * output mapping
 *    - RD0  -> RP64  -> RPOR0bits.RP64R
 *    - UART1 transmit-> 0b000001
 *
 * thus,
 *    RPINR18bits.U1RXR = 0b100 1011; // input map
 *    RPOR0bits.RP64R = 0b000001;     // output map
 *
 *  [ ] Configure the UART1 to work at 9600 bps.
 *  [ ] Read a character from UART1 and send it back to the UART1
*/
```

Step 1: gathering the ingredients

c

```c
#include <xc.h>

int uart1_setup(){
    U1STA = 0x00;   // reset control and status register

    U1STAbits.UTXISEL0 = 0;  // UARTx Transmission Interrupt Mode Selection bits
    U1STAbits.UTXISEL1 = 0;
    U1STAbits.UTXBRK = 0;   // UARTx Transmit Break bit
    U1STAbits.UTXEN = 0;    // UARTx Transmit Enable bit
    // U1STAbits.UTXBF - UARTx Transmit Buffer Full Status bit (read-only)
    // U1STAbits.TRMT  - Transmit Shift Register Empty bit (read-only)


    U1STAbits.URXISEL = 0;  // UARTx Receive Interrupt Mode Selection bits
    // U1STAbits.PERR  - Parity Error Status bit (read-only)
    // U1STAbits.FERR  - Framing Error Status bit (read-only)
    // U1STAbits.OERR  - Receive Buffer Overrun Error Status bit (read/clear only)
    // U1STAbits.URXDA - Receive Buffer Data Available bit (read-only)

    /*-------------------------------------------------------------------------------*/
    U1MODE = 0x00;  // reset mode register

    U1MODEbits.UARTEN = 0;  // UARTx Enable bit
    U1MODEbits.PDSEL = 0;   // Parity and Data Selection bits
    U1MODEbits.STSEL = 0;   // Stop Bit Selection bit
    U1MODEbits.BRGH = 0;    // High Baud Rate Enable bit

    /*-------------------------------------------------------------------------------*/
    U1BRG = 0;  // Baud
}

int uart1_blocking_read(){ // polling mode, read could (should) be implemented via *UART1 transimission
interrupt*
    return U1RXREG; // pulls a characterfrom the RX FIFO (read)
}

int uart1_blocking_write(int data_byte){
    U1TXREG = data_byte;    // pushes a character to the TX FIFO (write)
}

int main(){
    RPINR18bits.U1RXR = 0b1001011; // input map
    RPOR0bits.RP64R = 0b000001;    // output map

    uart1_setup();
    // send back every received character
    return 0;
}
```

Step 2: reframe

```c
#include <xc.h>

int uart1_setup(){
    U1STA = 0x00;   // reset control and status register
    U1MODE = 0x00;  // reset mode register
    U1BRG = 0;      // Baud

    /* UTXISEL<1:0> SETTINGS:
    If UTXISEL<1:0> = 00
        the UxTXIF is set when a character is transferred from the transmit
        buffer to the Transmit Shift register (UxTSR). This implies that at
        least one location is empty in the transmit buffer.
    If UTXISEL<1:0> = 01
        the UxTXIF is set when the last character is shifted out of the UxTSR
        register. This implies that all the transmit operations are completed.
    If UTXISEL<1:0> = 10
        the UxTXIF is set when the character is transferred to the UxTSR
        register and the transmit buffer is empty.*/

    U1STAbits.UTXISEL0 = 0;  // UARTx Transmission Interrupt Mode Selection bits
    U1STAbits.UTXISEL1 = 0;

    U1STAbits.UTXBRK  = 0;  // UARTx Transmit Break bit
    U1STAbits.UTXEN   = 0;  // UARTx Transmit Enable bit
    U1STAbits.URXISEL = 0;  // UARTx Receive Interrupt Mode Selection bits

    U1MODEbits.UARTEN = 0;  // UARTx Enable bit
    U1MODEbits.PDSEL  = 0;  // Parity and Data Selection bits
    U1MODEbits.STSEL  = 0;  // Stop Bit Selection bit
    U1MODEbits.BRGH   = 0;  // High Baud Rate Enable bit
}

int uart1_blocking_read(){ // polling mode, read could (should?) be implemented via *UART1 transimission
interrupt*
    // U1STAbits.PERR  - Parity Error Status bit (read-only)
    // U1STAbits.FERR  - Framing Error Status bit (read-only)
    // U1STAbits.OERR  - Receive Buffer Overrun Error Status bit (read/clear only)
    // U1STAbits.URXDA - Receive Buffer Data Available bit (read-only)

    return U1RXREG; // pulls a characterfrom the RX FIFO (read)
}

int uart1_blocking_write(int data_byte){
    // U1STAbits.UTXBF - UARTx Transmit Buffer Full Status bit (read-only)
    // U1STAbits.TRMT  - Transmit Shift Register Empty bit (read-only)


    U1TXREG = data_byte;    // pushes a character to the TX FIFO (write)
    return 0;
}

int main(){
    RPINR18bits.U1RXR = 0b1001011; // input map
    RPOR0bits.RP64R = 0b000001;    // output map

    uart1_setup();
    // send back every received character
    return 0;
}
```

Step 3: implementing a basic driver

c

```c
#include <xc.h>
#include "timer.h"

#define FCY 72000000UL
#define BAUDRATE 9600
#define BRGVAL ((FCY/BAUDRATE)/16)-1 // only for slow speed mode

int uart1_setup(){
    U1STA = 0x00;   // reset control and status register
    U1MODE = 0x00;  // reset mode register
    U1BRG = BRGVAL;         // Baud

    U1MODEbits.UARTEN = 1;  // First UARTx Enable bit...
    U1STAbits.UTXEN   = 1;  // ...then UARTx Transmit Enable bit

    // wait at least 1/9600 s before sending first char
    tmr_wait_ms(TIMER1, 0.1);
    return 0;
}

int uart1_bloking_read(){ // wait until a character is available, then returns it
    while(!U1STAbits.URXDA); // Wait until Data Available flag is 1
    return U1RXREG;
}

void uart1_bloking_write(int data_byte){ // wait until there is room in the buffer, then writes
    while(U1STAbits.UTXBF);  // Wait here as long as Transmit Buffer is Full
    U1TXREG = data_byte;
}

int main(){
    RPINR18bits.U1RXR = 0b1001011; // input map
    RPOR0bits.RP64R = 0b000001;    // output map

    uart1_setup();

    int data;
    while(1){
        if(U1STAbits.URXDA){
            data = uart1_bloking_read();
            uart1_bloking_write(data);
        }
    }
    return 0;
}
```

Step 4: implementing a nonblocking driver

c

```c
#include <xc.h>
#include "timer.h"

#define FCY 72000000UL
#define BAUDRATE 9600
#define BRGVAL ((FCY/BAUDRATE)/16)-1 // only for slow speed mode

#define UART_BUFFER_SIZE 64
#define TX_BUFFER_FULL -1
#define RX_BUFFER_EMPTY -1

int uart1_standard_setup(){
    U1STA = 0x00;   // reset control and status register
    U1MODE = 0x00;  // reset mode register
    U1BRG = BRGVAL; // Baud

    U1MODEbits.UARTEN = 1;  // First UARTx Enable bit...
    U1STAbits.UTXEN   = 1;  // ...then UARTx Transmit Enable bit

    // Enable RX interrupt
    IFS0bits.U1RXIF = 0; // clear the flag
    IEC0bits.U1RXIE = 1; // enable the interrupt

    // uart1_write() will enable the interrupt when needed
    IEC0bits.U1TXIE = 0;
    IFS0bits.U1TXIF = 0;

    tmr_wait_ms(TIMER1, 0.1);
    return 0;
}

struct circular_buffer { // circular buffer
    unsigned char data_buffer[UART_BUFFER_SIZE];
    volatile int idxW;  // The index we WRITE to
    volatile int idxR;  // The index we READ from
};

struct circular_buffer U1RX_buffer;
struct circular_buffer U1TX_buffer;

int uart1_write(const char *msg) {
    int i = 0;
    while (msg[i] != '\0') {
        int next_idxW = (U1TX_buffer.idxW + 1) % UART_BUFFER_SIZE;

        if (next_idxW == U1TX_buffer.idxR) return TX_BUFFER_FULL;
        // do not try to write 64+ bytes at once

        U1TX_buffer.data_buffer[U1TX_buffer.idxW] = msg[i];
        U1TX_buffer.idxW = next_idxW;
        i++;
    }

    // Only force the flag high if the hardware FIFO has room.
    // If it's full, the hardware is already working and will set the flag itself.
    if(!U1STAbits.UTXBF) IFS0bits.U1TXIF = 1;
    IEC0bits.U1TXIE = 1;

    return i;
}

void __attribute__((__interrupt__)) _U1TXInterrupt(void){
    // AFTER A BYTE IS COPIED IN THE TX BUFFER, at least one location is empty in the transmit buffer.
    IFS0bits.U1TXIF = 0;

    U1TXREG = U1TX_buffer.data_buffer[U1TX_buffer.idxR];
    U1TX_buffer.idxR = (U1TX_buffer.idxR+1)%UART_BUFFER_SIZE;

    if(U1TX_buffer.idxR == U1TX_buffer.idxW){
        IEC0bits.U1TXIE = 0;
    }
}

```

```c
void __attribute__((__interrupt__)) _U1RXInterrupt(void){
    IFS0bits.U1RXIF = 0;

    if(U1STAbits.OERR == 1)U1STAbits.OERR = 0;

    while(U1STAbits.URXDA){
        unsigned char received_byte = U1RXREG;
        int next_idxW = (U1RX_buffer.idxW + 1) % UART_BUFFER_SIZE;

        if(next_idxW != U1RX_buffer.idxR){
            U1RX_buffer.data_buffer[U1RX_buffer.idxW] = received_byte;
            U1RX_buffer.idxW = next_idxW;
        } else {
            // Software buffer is full, data discarded
        }
    }
}

int uart1_read_char(void) {
    if (U1RX_buffer.idxR == U1RX_buffer.idxW) {
        return RX_BUFFER_EMPTY;
    }

    unsigned char data = U1RX_buffer.data_buffer[U1RX_buffer.idxR];
    U1RX_buffer.idxR = (U1RX_buffer.idxR + 1) % UART_BUFFER_SIZE;

    return data;
}

int uart1_read_string(char *dest_buffer, int max_len) {
    int count = 0;
    while ((U1RX_buffer.idxR != U1RX_buffer.idxW) && (count < max_len - 1)) {
        dest_buffer[count] = U1RX_buffer.data_buffer[U1RX_buffer.idxR];
        U1RX_buffer.idxR = (U1RX_buffer.idxR + 1) % UART_BUFFER_SIZE;
        count++;
    }

    dest_buffer[count] = '\0';
    return count;
}

int main(){
    RPINR18bits.U1RXR = 0b1001011; // input map
    RPOR0bits.RP64R = 0b000001;    // output map

    uart1_standard_setup();

    int data;
    while(1){
        /* --- UART NONBLOCKING --- */

        data = uart1_read_char();
        if(data != RX_BUFFER_EMPTY){
            char echo_str[2];
            echo_str[0] = (char)data;
            echo_str[1] = '\0'; // Null terminator

            uart1_write(echo_str);
        }

        /* --- UART NONBLOCKING --- */

        // Other tasks
    }

    return 0;
}
```

Lesson 6: SPI

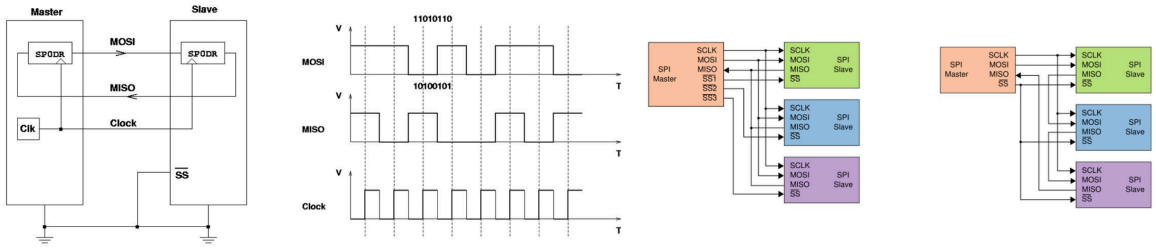
Overview

Serial Peripheral Interface (SPI) is a de facto standard (with many variants) for synchronous serial communication, used primarily in embedded systems for short-distance wired communication between integrated circuits.

Synchronous serial communication:
- The transmitter puts the data byte it wants to send into an internal shift register
- The transmitter uses a clock to shift the data bits out of its shift register onto an external data pin
- The receiver uses the input clock from the transmitter to sample incoming data
- The receiver puts the data on the input pin on an internal shift register.

Full-duplex communication:
- Two data lines are used: MISO (Master In Slave Out) and MOSI (Master Out Slave In)
- The master generates a clock signal and send it to the slave
- Both master and slave transmit data at the same time
- After transmission, the contents of the two shift registers are swapped.

Multi-node communication:
- A master can talk to many different slaves (a slave select signal may be used for guaranteeing that only one slave is enabled to write on the MISO line)
- Alternatively, nodes can be connected in a daisy-chain topology,



> ***[Note]*** **daisy chain configuration**: the first slave's output is connected to the second slave's input, and so on with subsequent slaves, until the final slave, whose output is connected back to the master's input. This effectively merges the individual communication shift registers of each slave to form a single larger combined shift register that shifts data through the chain. This configuration only requires a single SS line from the master, rather than a separate SS line for each slave.

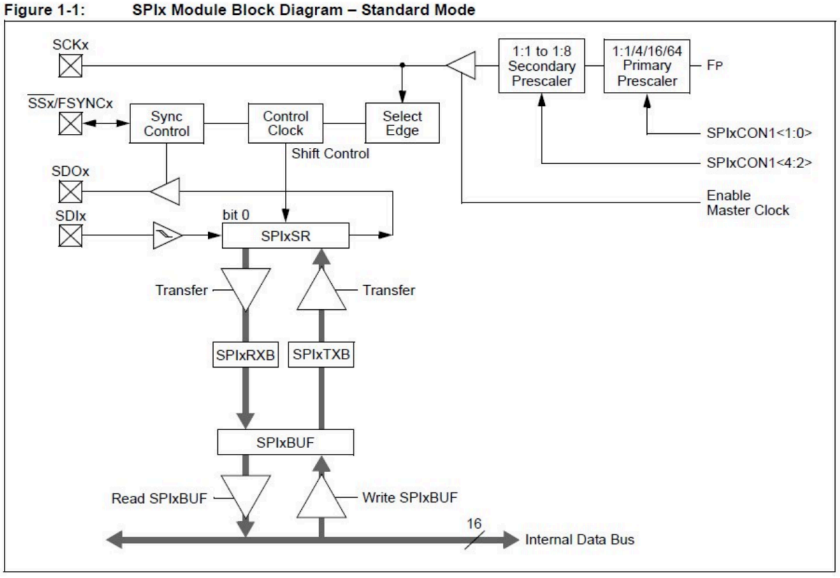
Main advantages over RS232	Main disadvantages over RS232
• High throughput (up to 70 Mbps) • High bandwidth • Transmission not limited to 8-bit words • Slaves use the master's clock, thus there is no need for precision oscillators • Multi-node communication available	• More expensive for clock generation circuitry • More pins and wires required • Supports only one master device at a given time • Less robust to noise • No flow control or parity check • Only handles short distances

Applications

SPI is used to talk to a variety of peripherals, such as:
- Sensors: IMU, temperature, magnetometer, pressure, ADC, touchscreens, video game controllers
- Control devices: audio codecs, digital potentiometers, DACs
- Camera lenses: Canon EF lens mount
- Memory: flash and EEPROMs
- Real-time clocks
- LCDs, sometimes even for managing image data
- Any MMC or SD card (including SDIO variant[note 3])
- Shift registers for additional I/O[8][9]

dsPIC33F SPI Module

Figure 1-1: SPIx Module Block Diagram – Standard Mode



SPIxBUF Register

The `SPIxTXB` and the `SPIxRXB` are unidirectional, 16-bit registers that share the SFR address of the `SPIxBUF` register. If the user application writes data to be transmitted to the `SPIxBUF` register, internally the data is written to the `SPIxTXB` register. Similarly, when the user application reads the received data from the `SPIxBUF` register, internally the data is read from the `SPIxRXB` register.

SPIxSTAT Register

Register 2-1: SPIxSTAT: SPIx Status and Control Register

R/W-0	U-0	R/W-0	U-0	U-0	R/W-0	R/W-0	R/W-0
SPIEN	—	SPISIDL	—	—	SPIBEC2(1)	SPIBEC1(1)	SPIBEC0(1)
bit 15							bit 8

R/W-0	R/C-0, HS	R/W-0	R/W-0	R/W-0	R/W-0	R-0, HS, HC	R-0, HS, HC
SRMPT	SPIROV	SRXMPT	SISEL2(1)	SISEL1(1)	SISEL0(1)	SPITBF	SPIRBF
bit 7							bit 0

`SPIxSTAT`: SPI Status registers, contains the flags that notify the program of certain events.

- `sSPIEN` needed to enable the SPI peripheral.
- `SPITBF` tells if the transmit buffer is full
- `SPIRBF` tells if the receiver buffer is full
- `SPIROV` tells if a receiver buffer overrun has occurred

SPIxCON1 Register

Register 2-2: SPIxCON1: SPIx Control Register 1

U-0	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	—	DISSCK(5)	DISSDO	MODE16	SMP(2)	CKE(1)
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
SSEN(4)	CKP	MSTEN	SPRE2(3)	SPRE1(3)	SPRE0(3)	PPRE1(3)	PPRE0(3)
bit 7							bit 0

- `MSTEN`: enables SPI master mode
- `MODE16`: selects 16/8 bits mode
- `PPRE`: primary prescaler
- `SPRE`: secondary prescaler
- `CKP` specifies the idle value of the clock

[!] SPI prescalers are different from timer prescalers!

SPI Overflow

If a new data word has been shifted into the SPIxSR register and the previous SPIxBUF register contents have not been read, the SPIROV bit (inside SPIxSTAT) is set. Any received data in the SPIxSR register is not transferred, and further data reception is disabled until the SPIROV bit is cleared. The SPIROV bit is not cleared automatically by the module; it must be cleared by the user application.

SPI Example

Example: writing one character to the SPI (Fcy = 2 MHz, SPI clock 400 KHz)

```c
#define MAG_CS LATDbits.LATD6 // pin associated to the Slave Select (SS)

unsigned int spi_write(unsigned int data) {
    while (SPI1STATbits.SPITBF == 1); // Wait until TX buffer is empty
    SPI1BUF = data;                   // Send the byte
    while (SPI1STATbits.SPIRBF == 0); // Wait until RX buffer is full
    return SPI1BUF;                   // Return the received byte
}

// Writing example: write 0x01 at 0x4B
MAG_CS = 0;
spi_write(0x4B); // MSB è già 0 -> non ho bisogno di azzerarlo manualmente
spi_write(0x01);
MAG_CS = 1;


// Reading example: read data from address 0x80
MAG_CS = 0;
spi_write(read_addr | 0x80); // MSB = 1 (must be done manually)
value_from_chip = spi_write(0x00); // sends zeroes to generate clock to retrieve data
MAG_CS = 1;
```

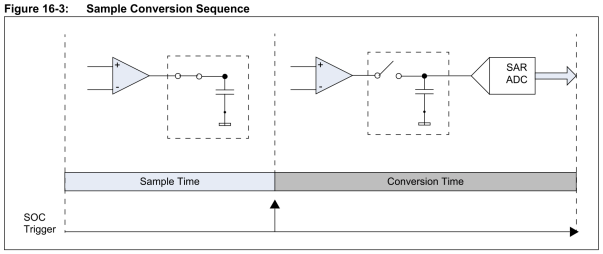
BOSCH BMX055 SPI interface

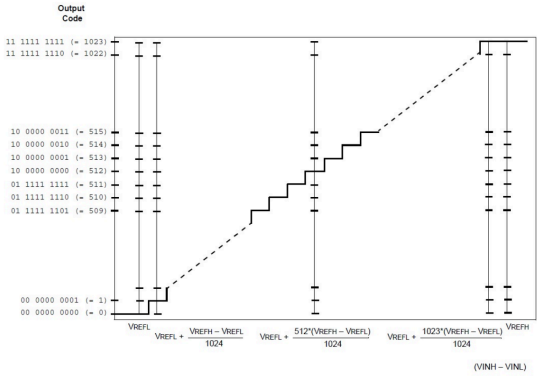
Small, versatile 9-axis sensor module

Lesson 7: ADC

Overview

Figure 16-3: Sample Conversion Sequence

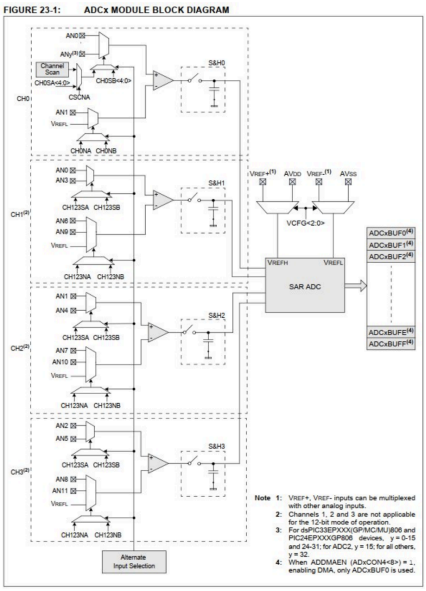




The analog-to-digital conversion constitutes a two-phase sequential process: **Sampling** (acquisition of the analog signal via the sample-and-hold amplifier) and **Conversion** (successive approximation translation into the digital domain)

dsPIC33 ADC Modules

FIGURE 23-1: ADCx MODULE BLOCK DIAGRAM



- Two independent ADC modules
- Up to 1 Msps conversion rate
- Selectable 10-bit or 12-bit operation modes
- Up to 32 analog pins (ADC 1)
- Up to 16 analog pins (ADC 2)
- 4 differential Sample/Hold (S/H) amplifiers (channels)
- 1 shared Successive Approximation converter
- Channel scan mode
- 16-word deep buffer for conversion results
- Selectable hardware or software conversion trigger sources

Timing Parameters

	Name	Description	Formula
T_{CY}	Instruction Cycle Period	The fundamental execution timebase of the microcontroller, derived from the system oscillator.	System defined
`TAD`	ADC Clock Period	The internal timebase strictly governing the ADC's successive approximation register (SAR) timing. It is a scaled derivative of T_{CY}.	`T_CY * (ADCS + 1)`
T_{SMP}	Sample Time	The temporal window during which the analog input channel is electrically coupled to the sample-and-hold (S&H) capacitor. Must exceed the hardware-defined minimum to ensure capacitor equilibrium.	Auto: `SAMC * TAD` Manual: SW defined
T_{CONV}	Conversion Time	The rigid hardware latency required to fully translate the captured analog voltage into a binary word, strictly dependent on hardware resolution.	10-bit: `12 * TAD` 12-bit: `14 * TAD`
F_{SAMP}	Sampling Rate	The absolute maximum throughput frequency of the ADC pipeline for a single channel.	`1 / (T_SMP + T_CONV)`

ADC Configuration Registers

The temporal configuration of the sampling and conversion phases is dictated by specific bitfields within the ADC Special Function Registers (SFRs):

- **`ADCS<7:0>`** (ADC Conversion Clock Select bits): A divider network parameter that scales the instruction cycle clock (T_{CY}) to synthesize the required `TAD` period.
- **`SAMC<4:0>`** (Auto-Sample Time bits): Determines the integer number of `TAD` periods allocated to the sampling phase (T_{SMP}) when auto-sampling is enabled.

> **[!] Hardware Timing Violations**
> The structural integrity of the analog-to-digital conversion strictly relies on meeting minimum time thresholds defined in the specific device's "Electrical Characteristics".
> - **Minimum `TAD`**: Accelerating the ADC clock beyond the silicon limit compromises the successive approximation process.
> - **Minimum T_{SMP}**: Insufficient sampling time prevents the internal holding capacitor from fully stabilizing at the external voltage level, causing aliasing and reading inaccuracies.

> ***[Note]* Design Objective:**
> For accurate A/D conversions, the physical hardware mandates a minimum `TAD` (e.g., 154 nsec for VDD = 5V). The firmware designer must computationally derive the correct `ADCS` prescaler to satisfy this absolute physical constraint before establishing the final sampling rate (F_{SAMP}).

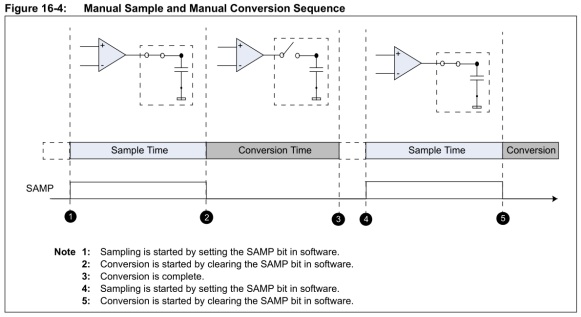
ADC Triggers

Sampling period **start**	Sampling period **end** (= conversion time start)
• automatic (at the end of previous conversion) • manual (by setting the `SAMP` bit in software)	• automatic (if T_{SMP} is fixed via the internal timer) • manual (by clearing the `SAMP` bit in software) • driven by other hardware events (e.g., Timers, external interrupts)

manual sample - manual conversion

Application Example: Reading a simple analog sensor exactly once triggered by a user action or a call inside a periodic task. Precise timing between readings is not critical, and software control is sufficient.

Figure 16-4: Manual Sample and Manual Conversion Sequence



Note 1: Sampling is started by setting the SAMP bit in software.
2: Conversion is started by clearing the SAMP bit in software.
3: Conversion is complete.
4: Sampling is started by setting the SAMP bit in software.
5: Conversion is started by clearing the SAMP bit in software.

```c
// --- ADC Setup Phase ---
AD1CON1bits.ASAM = 0;     // Manual sampling
AD1CON1bits.SSRC = 0b000; // Manual conversion trigger

// --- Execution Phase ---
// Start sampling
AD1CON1bits.SAMP = 1;

// Wait for sampling time (10us)
DelayUs(10);

// Start the conversion (ends sampling)
AD1CON1bits.SAMP = 0;

// Wait for the conversion to complete
while (!AD1CON1bits.DONE);

// Clear the conversion done flag
AD1CON1bits.DONE = 0;

// Read the conversion result
ADCValue = ADC1BUF0;
```

manual sample - automatic conversion

Application Example: Event-driven acquisitions where you want to start a measurement immediately after a specific software state, but cannot afford blocking `DelayUs()` loops. The hardware accurately enforces the minimum required sampling time, freeing the CPU to execute a few other instructions.

```c
// --- ADC Setup Phase ---
AD1CON1bits.ASAM = 0;     // Manual sampling
AD1CON1bits.SSRC = 0b111; // Auto-conversion (internal counter)
AD1CON3bits.SAMC = 16;    // Auto-Sample Time = 16 * TAD

// --- Execution Phase ---
// Manually initiate the sampling phase
AD1CON1bits.SAMP = 1;

// Hardware automatically waits for SAMC duration
// Hardware automatically clears SAMP and starts conversion

// Wait for the conversion to complete
while (!AD1CON1bits.DONE);

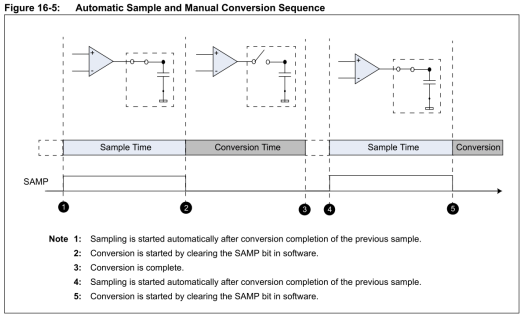
// Clear the conversion done flag
AD1CON1bits.DONE = 0;

// Read the conversion result
ADCValue = ADC1BUF0;
```

automatic sample - manual conversion

Application Example: Reading high-impedance sensors (e.g., specific environmental probes) that require exceptionally long charging times for the sample-and-hold capacitor. This pattern continuously charges the capacitor while the CPU manages a custom, long-duration delay.

Figure 16-5: Automatic Sample and Manual Conversion Sequence



Note 1: Sampling is started automatically after conversion completion of the previous sample.
2: Conversion is started by clearing the SAMP bit in software.
3: Conversion is complete.
4: Sampling is started automatically after conversion completion of the previous sample.
5: Conversion is started by clearing the SAMP bit in software.

```c
// --- ADC Setup Phase ---
AD1CON1bits.ASAM = 1;     // Auto-sampling (starts immediately)
AD1CON1bits.SSRC = 0b000; // Manual conversion trigger

// --- Execution Phase ---
while (1){
  // Wait for custom sampling time (100ms)
  DelayNmSec(100);

  // Start the conversion (ends sampling)
  AD1CON1bits.SAMP = 0;

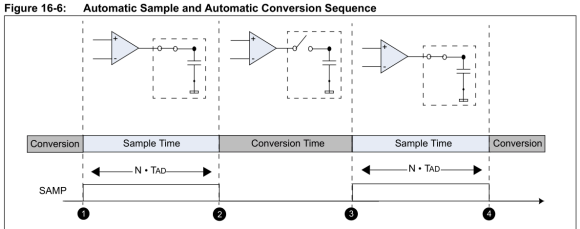
  while (!AD1CON1bits.DONE);
  AD1CON1bits.DONE = 0;
  ADCValue = ADC1BUF0;
}
```

automatic sample - automatic conversion

Application Example: Continuous digital signal processing, such as audio filtering, AC power waveform analysis, or direct memory streaming via DMA. The hardware automatically takes a sample, converts it, and immediately starts the next sample in an endless loop, without needing any CPU intervention.

- **Maximum Speed:** It operates independently of software instruction delays.
- **Perfect Timing (No Jitter):** The hardware guarantees perfectly equally-spaced samples.
- **Continuous Streaming:** It is optimal for applications requiring a high-speed flow of data.

Figure 16-6: Automatic Sample and Automatic Conversion Sequence



Note 1: Sampling starts automatically after conversion.
2: Conversion starts automatically upon termination of self timed sampling period.
3: Sampling starts automatically after conversion.
4: Conversion starts automatically upon termination of self timed sampling period.

```c
// --- ADC Setup Phase ---
AD1CON1bits.ADON = 0;     // Disable ADC during configuration

// 1. Configure Pin (e.g., AN0)
TRISAbits.TRISA0 = 1;     // Set pin as digital input
ANSELAbits.ANSA0 = 1;     // Enable analog pin mode
AD1CHS0bits.CH0SA = 0;    // Route AN0 to Channel 0

// 2. Configure Timing
AD1CON3bits.ADCS = 10;    // Set TAD clock prescaler
AD1CON3bits.SAMC = 15;    // Auto-Sample Time = 15 * TAD

// 3. Enable Auto-Auto Mode
AD1CON1bits.ASAM = 1;     // Trigger: Auto-Sample immediately
AD1CON1bits.SSRC = 0b111; // Trigger: Auto-Convert (internal counter)

// Turn ADC back on
AD1CON1bits.ADON = 1;

// --- Execution Phase ---
while (1){
    // Wait for hardware to finish both phases
    while (!AD1CON1bits.DONE);

    // Clear the conversion done flag
    AD1CON1bits.DONE = 0;

    // Read the conversion result
    ADCValue = ADC1BUF0;
}
```

ADC Multiple Channels

> ***[Note]*** check Simultaneous Sampling and Sequential Sampling

`ADxCON1` Register

REGISTER 23-1: ADxCON1: ADCx CONTROL REGISTER 1

R/W-0	U-0	R/W-0	R/W-0	U-0	R/W-0	R/W-0	R/W-0
ADON	—	ADSIDL	ADDMABM	—	AD12B(1)	FORM<1:0>	
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0, HSC	R/C-0, HSC
SSRC<2:0>			SSRCG	SIMSAM	ASAM(3)	SAMP	DONE(3)
bit 7							bit 0

`ADxCON1`: ADC Control register 1.

- `ADON`: turns on the ADC module
- `SSRC`: selects how the conversion should start (0 = manual, 7 = internal counter)
- `ASAM`: selects how the sampling should start
- `SAMP`: sample enable bit
- `DONE`: tells when the A/D conversion is done

`ADxCON2` Register

REGISTER 23-2: AD1CON2: ADC1 CONTROL REGISTER 2

R/W-0	R/W-0	R/W-0	U-0	U-0	R/W-0	R/W-0	R/W-0
VCFG<2:0>			—	—	CSCNA	CHPS<1:0>	
bit 15							bit 8

R-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
BUFS	SMPI<4:0>					BUFM	ALTS
bit 7							bit 0

`ADxCON2`: ADC Control register 2:

- `CSCNA`: selects if inputs should be scanned
- `CHPS`: selects the channels
- `SMPI`: after how many conversions when the IF should be set to 1

ADxCON3 Register

REGISTER 23-4: ADxCON3: ADCx CONTROL REGISTER 3

R/W-0	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
ADRC	—	—	SAMC<4:0>(1)				
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
ADCS<7:0>(2,3)							
bit 7							bit 0

ADxCON3: ADC Control register 3:

- SAMC: how long the sampling should last [0 Tad - 31 Tad] (it matters only if SSRC = 7)
- ADCS: selects how long is one Tad [1 Tcy - 64 Tcy]

ADxCHS0 Register

REGISTER 23-7: ADxCHS0: ADCx INPUT CHANNEL 0 SELECT REGISTER

R/W-0	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CH0NB	—	—	CH0SB<4:0>(1)				
bit 15							bit 8

R/W-0	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CH0NA	—	—	CH0SA<4:0>(1)				
bit 7							bit 0

ADxCHS0: selects the inputs to channel 0

ADxCHS123 Register

REGISTER 23-6: ADxCHS123: ADCx INPUT CHANNEL 1, 2, 3 SELECT REGISTER

U-0	U-0	U-0	U-0	U-0	R/W-0	R/W-0	R/W-0
—	—	—	—	—	CH123NB<1:0>		CH123SB
bit 15							bit 8

U-0	U-0	U-0	U-0	U-0	R/W-0	R/W-0	R/W-0
—	—	—	—	—	CH123NA<1:0>		CH123SA
bit 7							bit 0

ADxCHS123: selects the inputs to channels 1, 2 and 3 Results will be found in registers ADCxBUF0 through ADCxBUFF 1

Other Registers

Register 16-7: AD1CSSH: ADC1 Input Scan Select Register High

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CSS31	CSS30	CSS29	CSS28	CSS27	CSS26	CSS25	CSS24
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CSS23	CSS22	CSS21	CSS20	CSS19	CSS18	CSS17	CSS16
bit 7							bit 0

Register 16-8: ADxCSSL: ADCx Input Scan Select Register Low

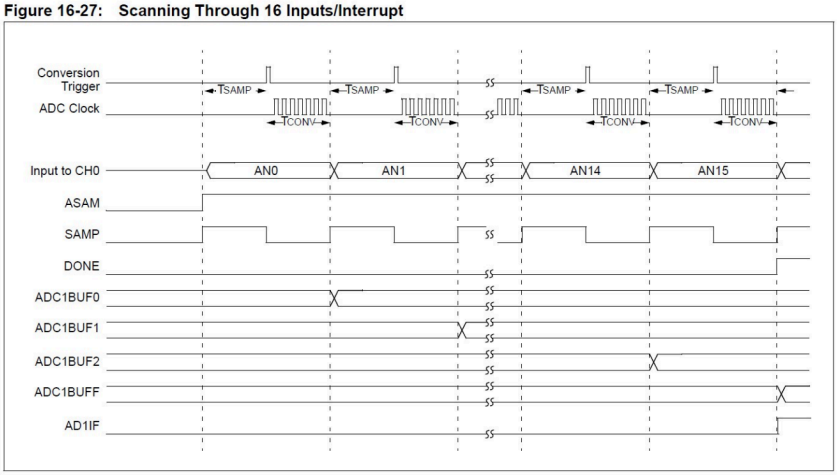
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CSS15(3)	CSS14(3)	CSS13(3)	CSS12	CSS11	CSS10	CSS9	CSS8
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CSS7	CSS6	CSS5	CSS4	CSS3	CSS2	CSS1	CSS0
bit 7							bit 0

1 = scan, 0 = do not scan

Scan Mode

Figure 16-27: Scanning Through 16 Inputs/Interrupt



`ANSELx` Register

`ANSELx`: sets pins as analog (1) or digital (0). As a thumb rule, always set each ANSELx to 0 and then toggle to 1 only the pins required to be analog

These registers are important, failing to configure them will result in errors during the conversion

Configuring the ADC

1. select your Tad ADC time and set `ADxCON3bits.ADCS` (suggested value is 8 for our board)
2. select how sampling begins
   - manual: `ADxCON1bits.ASAM = 0;`
   - automatic: `ADxCON1bits.ASAM = 1;`
3. select how sampling ends and conversion begins
   - manual: `ADxCON1bits.SSRC = 0;`
   - automatic: `ADxCON1bits.SSRC = 7;` and then choose how long sampling lasts by setting `ADxCON3bits.SAMC`;
4. choose how many channels you want to use setting `ADxCON2bits.CHPS`
5. choose the positive input to the channels by setting `ADxCHS0bits.CH0SA`; and `ADxCHS123bits.CH123SA;`
6. set the input pins as analog (value 1) in `ANSELx` and the rest as digital (value 0)
7. if more than 1 channel is used, choose sequential or simultaneous sampling using `AxDCON1bits.SIMSAM`
8. if sequential set `ADxCON2bits.SMPI`; to number of inputs - 1; if simultaneous consider the number of channels used
9. finally, turn ADC on with `ADxCON1bits.ADON = 1;`

DONE bit

The DONE bit is only cleared at the start of each conversion and is set at the completion of the conversion, but remains set indefinitely, even through the next sample phase until the next conversion begins.

If application code is monitoring the DONE bit in any kind of software loop, the user must consider this behavior because the CPU code execution is faster than the ADC.

As a result, in Manual Sample mode, particularly where the user's code is setting the SAMP bit, the DONE bit should also be cleared by the user application just before setting the SAMP bit.

Appendix 1: Installation of the required Tools

The tools required to compile and flash programs onto the microcontroller are:

- **Mikrobootloader**: specifically the `mikroe-uhb` Linux utility.
- **MPLAB for Visual Studio Code**: (Alternatively, the standalone MPLAB X IDE can be used).

Install the Mikrobootloader

The mikroE mikroBootloader software is required to download the firmware onto the board. mikroe-uhb GitHub repository.

Install MPLAB in VS Code

In the VS Code Extensions Marketplace it is required to install the MPLAB extension pack. For detailed instructions, refer to the MPLAB for Visual Studio Code official guide.

Appendix 2: Working Pipeline

Step 1: Create a Workspace and Project

- **Create a workspace folder** and open VS Code in it (run: `code .` from the terminal)
- **Create a new project:** Inside VS Code, open the Command Palette (`Ctrl + Shift + P`) and:
  1. Select `> MPLAB: Create New Project`
  2. Enter the **Project name**
  3. Enter the **Project folder** (A new folder for each project in the workspace keeps the structure clean)
  4. Select board: `dsPIC33EP512MU810`
  5. Select compiler: `XC16`

Step 2: Switch from Standard Desktop C environment to Embedded C environment

By default, VS Code assumes the user is writing standard desktop C code and uses the system's default GCC compiler. For this reason IntelliSense produces the "file not found" errors when it encounters embedded-specific libraries (like `#include <xc.h>`) and fails to recognize microcontroller hardware registers.

To fix this, it is required to create/modify the configure VS Code's `c_cpp_properties.json` file to act as an embedded IDE.

By linking the Microchip XC16 compiler paths and defining our specific dsPIC33 chip model, IntelliSense is enabled to read Microchip's libraries. This eliminates false errors and enables correct syntax highlighting and auto-completion for embedded development.

In short, it must be added to the json file the path to:

- `xc16/vX.XX/support`
- `xc16/vX.XX/support/dsPIC33E/h`

That, in Ubuntu, are:

- `/opt/microchip/xc16/v2.10/include`
- `/opt/microchip/xc16/v2.10/support/dsPIC33E/h`

Standard Desktop C environment	Embedded C environment

<pre>{
  "configurations": [
    {
      "name": "Linux",
      "includePath": [
        "${workspaceFolder}/**"
      ],
      "defines": [],
      "compilerPath": "/usr/bin/gcc",
      "cStandard": "c17",
      "cppStandard": "gnu++17",
      "intelliSenseMode": "linux-gcc-x64"
    }
  ],
  "version": 4
}</pre>
	<pre>{
	  "configurations": [
	    {
	      "name": "Linux",
	      "includePath": [
	        "${workspaceFolder}/**",
	        "/opt/microchip/xc16/v2.10/include",
	        "/opt/microchip/xc16/v2.10/support/dsPIC33E/h"
	      ],
	      "defines": [
	        "__dsPIC33EP512MU810__"
	      ],
	      "compilerPath": "/opt/microchip/xc16/v2.10/bin/xc16-gcc",
	      "cStandard": "c99",
	      "intelliSenseMode": "gcc-x64"
	    }
	  ],
	  "version": 4
	}</pre>

Example of workspace:

```
ric@ububu:~/Desktop/Embedded System/Assignments$ tree
.
└── baseTemplate
    ├── _build
    ├── cmake
    ├── main.c
    ├── out
    │   └── baseTemplate
    │       ├── default.elf
    │       ├── default.hex
    │       └── production
    │           ├── default-production.elf
    │           └── default-production.hex
    └── README.md

41 directories, 40 files
```

Step 3: Build the Project

Once the code is ready, before flashing it into the uC, it must be Build to generate the `.hex` file.

It can be done either, inside VS Code, via:

- the **Build** icon (the hammer) in the MPLAB sidebar,
- the Command Palette:
  `> MPLAB: Build Project`
  This will generate your `production.hex` file.

Step 4: Get product and vendor id

> ***[Note]*** Whenever the board is connected or the reset button is pushed the on-board mantain a 5-second state in wich is ready and waits for an upstream of new data to be copied in the flash memory

```
ric@ububu:~$ sudo lsusb
...
Bus 001 Device 008: ID 2dbc:0001 Mikroelektronika USB HID Bootloader
...
```

We are interested into the `Mikroelektronika USB HID Bootloader`

- Vendor=`0x2dbc`
- Product=`0x0001`

Step 5: Flash the Program onto the uC

After building it is possible to flash the binary file on the uC. The `production.hex` file can be found at `ProjectWorkspace/out/baseTemplate/production/default-production.hex`

```
sudo mikroe-uhb -v --vendor=0x2dbc --product=0x0001 PATH-TO-production.hex
```

The bootloader allows the flashing of the new firmware only in that 5 seconds window after turning on the uC so, once the flash command has been run, it might be necessary to reboot the board